

TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY
BHAKTAPUR MULTIPLE CAMPUS

A Project Report on

"careerNexus"

In partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Information Technology

Under the supervision of

Mr. Bishwa Nath Lal Karn

Department of CSIT

Bhaktapur Multiple Campus

Submitted By:

Sarthak Ghimire (28331/078),

Sujal Limbu (28339/078),

Suraj Gautam (30222/078)

Submitted To:

Institute of Science and Technology

Tribhuvan University

Kirtipur, Nepal

2082-07-18



SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project prepared under my supervision by Mr. Sarthak Ghimire, Mr. Sujal Limbu and Mr. Suraj Gautam entitled "careerNexus" in partial fulfillment of the requirement for the degree of the degree BSc. in Computer Science and Information Technology (BSc. CSIT) be processed for evaluation.

……

Supervisor

Mr. Bishwa Nath Lal Karn

Bhaktapur Multiple Campus



CERTIFICATE OF APPROVAL

This is to certify that the project entitled "careerNexus" prepared by Mr. Sarthak Ghimire (28331/078), Mr. Sujal Limbu (28339/078) and Mr. Suraj Gautam (30222/078) in partial fulfillment of the requirements for the four years Bachelor Degree in Bachelor of Science in Computer Science & Information Technology has been well studied. In our opinion, it is satisfactory in the scope and quality of a project for the required degree.

……	……
HOD/Coordinator	Supervisor
Mr. Sushant Paudel	Mr. Bishwa Nath Lal Karn
Bhaktapur Multiple Campus	Bhaktapur Multiple Campus

……	……
Internal Examiner	External Examiner
Bhaktapur Multiple Campus	IOST, Tribhuvan University

ACKNOWLEDGEMENT

We owe our most profound appreciation to Bhaktapur Multiple Campus for giving us a chance to work on this project as part of our syllabus.

We are highly thankful to our supervisor, Mr. Bishwa Nath Lal Karn for his consistent support, guidance and feedback throughout the report's creation. We are generously obligated to him for providing this excellent opportunity to expand our knowledge. It helped a lot to realize what we study for.

We want to sincerely thank our respected coordinator, Mr. Sushant Paudel. We are thankful to all the faculty members and lecturer/teachers who helped us, directly or indirectly, in completing this project.

We would also like to extend our appreciation to our team members for their hard work and dedication in completing this project.

Thank You,

Sarthak Ghimire(28331/078)

Sujal Limbu(28339/078)

Suraj Gautam(30222/078)

ABSTRACT

careerNexus is an intelligent job portal built on the MERN stack that bridges the gap between job seekers and employers through AI-powered resume-job matching. The platform supports three modes: job seekers, employers, and admin dashboard, each tailored for specific roles and access. It uses cosine similarity to match resumes with relevant job listings based on skills, experience, and keywords. The system offers dynamic job recommendations, efficient filtering and a smart search mechanism to enhance user experience. Administrators can oversee user activity, manage job listings, and monitor hiring trends through an intuitive dashboard. careerNexus brings automation, personalization, and intelligence to the job hunting and hiring process.

Keywords: job portal, MERN stack, resume-job matching, cosine similarity, admin dashboard, job seeker, employer, intelligent hiring system.

TABLE OF CONTENTS

SUPERVISOR'S RECOMMENDATION ..... **i**

CERTIFICATE OF APPROVAL ..... **ii**

ACKNOWLEDGEMENT ..... **iii**

ABSTRACT ..... **iv**

TABLE OF CONTENTS ..... **v**

LIST OF ABBREVIATIONS ..... **viii**

LIST OF FIGURES ..... **ix**

LIST OF TABLES ..... **xi**

CHAPTER 1:INTRODUCTION **1**

1.1 Introduction 1

1.2 Problem Statement 2

1.3 Objectives 2

1.4 Scopes and Limitations

1.4.1 Scopes 2

1.4.2 Limitations 2

1.5 Development Methodology 3

1.6 Report Organization 4

CHAPTER 2:BACKGROUND STUDY AND LITERATURE REVIEW **6**

2.1 Background Study 6

2.2 Literature Review 7

CHAPTER 3:SYSTEM ANALYSIS **8**

3.1 Requirement Analysis 8

3.1.1 Functional Requirements 8

3.1.2 Non-functional Requirements....10
3.2 Feasibility Study....10
3.2.1 Technical Feasibility....10
3.2.2 Operational Feasibility....11
3.2.3 Economic Feasibility....11
3.3 Analysis....12
3.3.1 Data Modelling: Entity-Relationship (ER) Diagram....12
3.3.2 Process Modelling: Data Flow Diagram (DFD)....13

CHAPTER 4:SYSTEM DESIGN....16
4.1 Architectural Design....16
4.2 Database Schema Design....16
4.3 Process Design....18
4.3.1 Login/ Registration Flowchart....18
4.3.2 Resume Matching Flowchart....19
4.3.3 Job Posting Flowchart....20
4.3.4 Job Application Submission Flowchart....21
4.4 Forms and Interface Design....22
4.4.1 Form Design....22
4.4.2 Interface Design....23
4.5 Algorithm Details....25

CHAPTER 5:IMPLEMENTATION AND TESTING....27
5.1 Implementation....27
5.1.1 Tools Used....27
5.1.2 Implementation Details of Modules....29
5.2 Testing....30
5.2.1 Unit Testing....30

5.2.2 System Testing..... 34
5.3 Algorithm Analysis……36
5.3.1 Login and Registration.....36
5.3.2 Resume Upload and AI Matching.....36
5.3.3 Job Posting and Searching..... 38

CHAPTER 6 :CONCLUSION AND FUTURE RECOMMENDATIONS.....39
6.1 Conclusion..... 39
6.2 Future Recommendations..... 40

REFERENCES.....41
APPENDICES

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
ATS	Applicant Tracking System
DFD	Data Flow Diagram
DOCX	Document
ER	Entity-Relationship
IDE	Integrated Development Environment
MERN	MongoDB, Express.js, React.js, Node.js
PDF	Portable Document Format
SPA	Single Page Application
UI/UX	User Interface/User Experience

LIST OF FIGURES

Figure 1: Waterfall Model....4
Figure 2: Use Case Diagram....9
Figure 3: ER Diagram....12
Figure 4: Level 0 DFD....14
Figure 5: Level 1 DFD....15
Figure 6: Architectural Design....16
Figure 7: Database Schema Design....17
Figure 8: Login/Registration Flowchart....18
Figure 9: Resume Matching Flowchart....19
Figure 10: Job Posting Flowchart....20
Figure 11: Application Submission Flowchart....21
Figure 12: Basic Layout of Forms....22
Figure 13: Interface for Sign In Form....23
Figure 14: Interface for Sign Up form....23
Figure 15: Interface for Dashboard....24
Figure 16: Interface for Social Feed....24
Figure 17: Sign In Form....40
Figure 18: Sign Up Form....40
Figure 19: Employer Dashboard....41
Figure 20: Company Dashboard....41
Figure 21: Manage Jobs Section....42
Figure 22: Resume Matching....42
Figure 23: Social Feed....43
Figure 24: Company Profile....43
Figure 25: Employee Dashboard....44
Figure 26: Job Browsing....44
Figure 27: Applying to a Job....45

Figure 28: Jobs Applied by a User.....45
Figure 29: User Resumes.....46
Figure 30: Community Directory.....46
Figure 31: User Profile.....47
Figure 32: Database Collection.....47
Figure 33: Jobs Collection.....48
Figure 34: Posts Collection.....48
Figure 35: Resume Collection.....49
Figure 36: User Collection.....49

LIST OF TABLES

Table 1: Tools Used in Project..... 27

Table 2: Test Case for User Registration..... 29

Table 3: Test Case for User Login..... 31

Table 4 : Test Case for Job Posting..... 31

Table 5 : Test Case for Job Searching..... 32

Table 6: Test Case For Application Submission..... 32

Table 7: Test Case For Employer Dashboard..... 33

Table 8: System Testing..... 33

Table 9: TF-IDF Vector Weights for Cosine Similarity Calculation..... 36

CHAPTER 1:INTRODUCTION

1.1 Introduction

Finding the right job or hiring the right person has always been a tough task in the job market. People often struggle to get the job that fits their skills, and companies waste a lot of time going through applications that don't match what they need. Old job websites usually just let users search with keywords, but they don't really understand what a person can do or what a company truly wants. This leads to a lot of wrong job suggestions, missed chances, and slow hiring processes.

With more and more people looking for better ways to connect with jobs, **careerNexus** comes in as a smart, modern job portal powered by artificial intelligence. It is built using the **MERN stack**: MongoDB, Express.js, React.js, and Node.js, which makes the system fast, easy to use, and able to grow as more users join.

The platform serves three main types of users:

- **Job Seekers**: People looking for work can create profiles, upload resumes, search for jobs, and apply with just a few clicks.
- **Employers**: Companies can post job openings, view applicant profiles, and manage applications all in one place.
- **Administrators**: They keep everything running smoothly by approving job posts, managing user accounts, and fixing any issues.

What makes careerNexus different is its AI-based matching system. Instead of just matching words, it reads the content of resumes and job descriptions to find real connections, like matching a developer who knows Python with a company looking for someone with coding and problem-solving skills. This helps job seekers find better opportunities and helps employers find the right talent faster.

The system also includes features like real-time notifications, a clean and simple design, and secure login, making the whole experience smooth for everyone. By bringing smart technology into job searching and hiring, *careerNexus* aims to save time, reduce frustration, and create better matches between people and jobs.

1.2 Problem Statement

Traditional job portals have a poor resume-job relevance with no personalized job recommendations, which results in irrelevant job recommendations for candidates. Due to limited job filtering capabilities, employers find it difficult to identify most suitable candidates from the large pool of applicants. Basic keyword-based searches fail to understand the context, which might lead to inefficient job discovery for users. Moreover, there is a lack of automation in analysing resumes based on skills and experience in traditional job portals, making them inefficient in the modern world. That is where careerNexus comes into play. It uses cosine similarity AI algorithm which helps in semantic matching of resumes and job descriptions.

1.3 Objectives

The main objectives of this project are listed below:

- To build a web based job portal to personalize job recommendation based on user profile.
- To implement role based access control and the cosine similarity algorithm to recommend jobs.

1.4 Scopes and Limitations

1.4.1 Scopes

- Users can create profiles, upload resumes, and browse or apply for jobs.
- Employers can post job openings and filter applicants using cosine similarity for matching.
- The system matches resumes and job listings using cosine similarity for relevance.
- Personalized job suggestions are shown to job seekers based on skills and experience.
- Admins can monitor activities, manage users, and control job listings.
- Notification system alerts users about new jobs, applications, and updates.

1.4.2 Limitations

- The system requires stable internet connectivity for real-time features.
- Resume matching accuracy depends on resume formatting and keyword clarity.

- It does not support video interviews or built-in chat between users.
- AI recommendations may not be 100% accurate for every user.
- Currently, only English-language resumes and listings are supported.

1.5 Development Methodology

We developed careerNexus using the traditional waterfall model.

The Waterfall Methodology for software development

careerNexus will be built based on the waterfall model. It is a traditional project management approach that follows a sequential, linear progression through different phases of a project. The model is suitable for projects with well-defined requirements and minimal changes.

Phases of Waterfall Methodology:

Requirements Gathering: Functional and non-functional requirements of the intelligent job portal were thoroughly studied. The features like resume-job matching were studied and problems were identified.

System Design: Architectural design of the MERN based system was created. MongoDB schemas, ExpressJS routes, ReactJS components and NodeJS services for backend logic were outlined. DFDs and ER diagrams were prepared to demonstrate interaction between different modules.

Implementation: All the modules of careerNexus were coded and integrated. Frontend interfaces were built using ReactJS, backend functionalities were coded in NodeJS and Express with MongoDB for data persistence. The Cosine Similarity Algorithm was implemented to match resumes with jobs.

Testing: Initially, unit testing was carried out to different units such as user registration, user login, job posting and job searching. Then, system testing was carried after integration of all the units to ensure their smooth functioning.

Deployment: The fully tested system is deployed and made operational in the production environment.

Maintenance: Scalability of the site is discussed to be improved in the upcoming maintenance sessions.

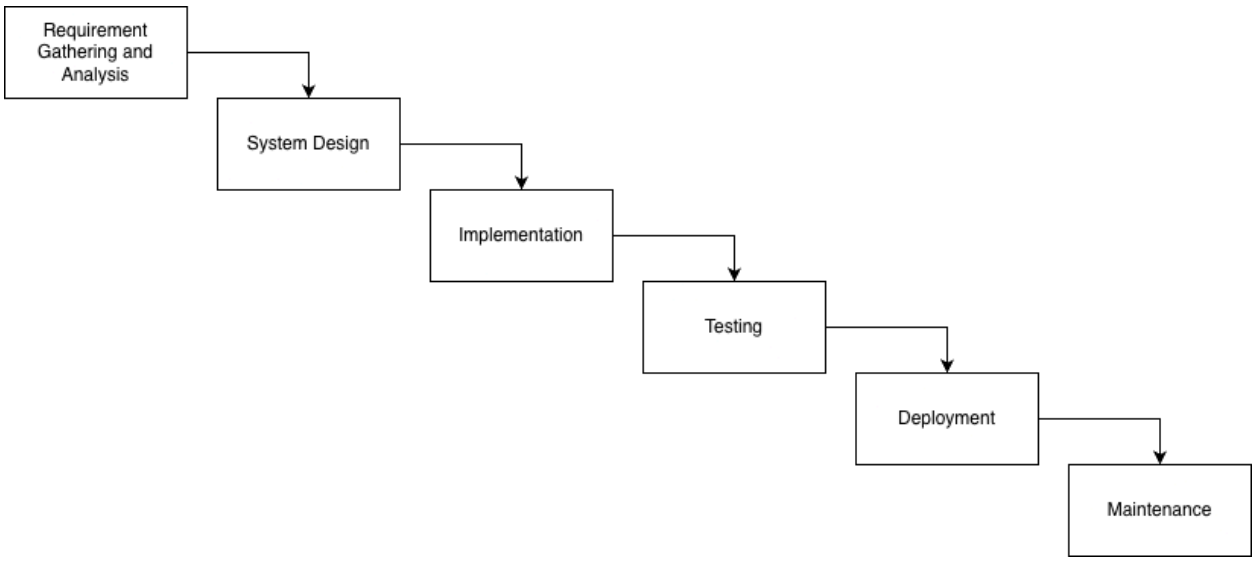


Figure 1: Waterfall Model

1.6 Report Organization

The project report is organized in such a way that it provides a logical flow of information to the readers.

Chapter 1 Introduction

Shows the background and overview, problem statement, objectives, scope and limitations of the project. Problem statement highlights the project for which the project was developed. This section is theoretical, and research based. The project objectives, scope and limitations are discussed.

Chapter 2 Background Study and Literature Review

Defines and describes Background Study and literature review about existing systems along with their advantages and disadvantages.

Chapter 3 System Analysis

Presents the System Analysis including Requirement Analysis which explains the Functional and Non-Functional Requirements of the project and Feasibility Analysis explains why/how the project is implemented.

Chapter 4 System Design

Presents the System Design including Database design diagram, Process design which includes flowchart for major components, Interface and Form design showcasing the overall UI/UX of the project.

Chapter 5 Implementation and Testing

Presents the Implementation and Testing, it clarifies the system workflow, and provides an indication of how the system is implemented, and what tools/platforms have been used.

Chapter 6 Conclusion and Future Recommendations

Presents the conclusion and recommendation marking an end to the documents by summing up the entire project and by opening a door for future research areas. This chapter summarizes the findings, discusses the project's implications, and suggests future recommendations.

CHAPTER 2:BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Online job portals have transformed the recruitment landscape, making the process far more efficient than the old-school methods of hiring. Back in the day, companies relied on newspaper ads, walk-in applications, or word-of-mouth, which often dragged on for weeks and came with hefty costs. But with digital platforms, things sped up dramatically, faster resume screening, less bias in selections thanks to algorithms, and overall savings on the entire hiring cycle. Monster.com, launched in the late 1990s after a merger of earlier sites, started this revolution by creating the first major online hub for job seekers and employers. Today, there are countless options out there, from giants like LinkedIn for professional networking to Indeed for broad job searches and Glassdoor for company insights. These sites have made it easier for anyone to apply from anywhere, leveling the playing field a bit.

As technology kept evolving, the explosion of AI in recent years has taken recruitment to another level, simplifying what used to be a tedious ordeal. AI tools are now baked into human resources practices, like machine learning-powered applicant tracking systems (ATS) that scan resumes in seconds. Predictive analytics, for instance, have slashed hiring times by up to 50%, allowing recruiters to focus on quality matches rather than sifting through piles of applications. It's not just about speed; AI helps reduce unconscious biases by prioritizing skills and experience over subjective factors, making the whole process feel more fair and streamlined for everyone involved.

COVID-19 pandemic only accelerated these shifts, pushing job recruitment almost entirely online and leaving paper-based systems in the dust. With lockdowns and remote work becoming the norm, companies had no choice but to go with virtual interviews, digital onboarding, and AI-driven tools to keep hiring going. This is where careerNexus comes in, it was designed to build on these advancements, using cosine similarity algorithm to match resumes with job postings more precisely. By doing so, it optimizes

the hiring flow, cutting down time and costs for both employers scouting talent and job seekers chasing opportunities.

2.2 Literature Review

Cosine similarity plays a central role in enhancing resume matching for job portals like careerNexus by effectively capturing semantic relationships between job descriptions and candidate profiles [1]. It measures the cosine of the angle between two vectors, offering a normalized metric that excels in high-dimensional text analysis, making it suitable for comparing diverse resume formats [2].

Research highlights its superiority over Jaccard similarity and Euclidean distance in recommendation systems, as it focuses on directional alignment rather than set overlap or magnitude differences [3]. Validation studies affirm that a 5-gram cosine similarity approach is viable for longer texts, such as resumes, by accounting for word order and providing deeper semantic insights compared to unigram methods [4]. This metric supports automated screening by quantifying resume-job alignment, reducing manual effort for recruiters [2].

In careerNexus, cosine similarity enables personalized job recommendations, leveraging machine learning to analyze candidate skills and preferences [1]. Its robustness is further validated through incremental studies, showing consistent performance across textual similarity tasks [4]. By integrating with resume parsing and predictive analytics, it enhances matching accuracy and streamlines the hiring process [3]. Overall, cosine similarity's ability to handle semantic nuances positions it as a key driver for transforming traditional job portals into efficient, platforms like careerNexus [1].

CHAPTER 3:SYSTEM ANALYSIS

Introduction:

The system analysis phase serves as the bedrock for successful software development, providing a systematic approach to understanding, planning, and defining the scope of the careerNexus project. Adopting the Waterfall Model as our chosen methodology, the system analysis phase unfolds as a structured, sequential process, emphasizing a meticulous progression through each stage. This phase commences with a thorough exploration of existing Job Portal Systems, seeking inspiration and identifying unique features that can enhance the user experience and operational efficiency. The Waterfall Model's linear progression ensures a well- defined path, allowing for in-depth investigation and comprehensive insights before moving to subsequent phases.

3.1 Requirement Analysis

Requirement analysis is the first step to building any project. It is a key element in the Waterfall Model which was initiated by thoroughly examining existing job portals. Insights gained from studying these systems identified unique features that inspired the design of careerNexus. Additionally, we found that previously available systems were complex to use and operate, which highlighted the need of simplicity in the user interface as well as overall functionality. Within the Waterfall Model's systematic framework, the requirement analysis phase stands as a critical juncture where the blueprint of careerNexus takes shape. This phase follows a study of existing similar systems, serving as the stepping stone in translating user expectations into concrete project requirements.

3.1.1 Functional Requirements

a) **User Registration and Login:** The system must allow job seekers and employers to create accounts and login using secure credentials.
b) **Job Posting:** Employers must be able to post job listings with job details.(Title, description, requirements and deadlines)
c) **Resume Upload:** Job seekers must be able to upload their resumes in formats like PDF and DOCX in order to match and store them.

d) **Job Search Functionality:** Users must be able to search jobs using keywords.

e) **Resume-Job Matching:** The system must use the cosine similarity algorithm to match the uploaded resume with relevant job posting in order to recommend the best fits to the employers.

f) **Application Submission:** Job seekers must be able to apply for jobs by submitting their profiles through the portal.

g) **Employer Dashboard:** Employers must have a dashboard to view, manage and shortlist candidates.

h) **Notification System:** The system must send notifications (email, app-alerts) to users about the status of their applications, interview schedules, job schedules, etc.

i) **Admin Management:** The administrators must be able to oversee user accounts, approve job postings as well as maintain the overall system settings.

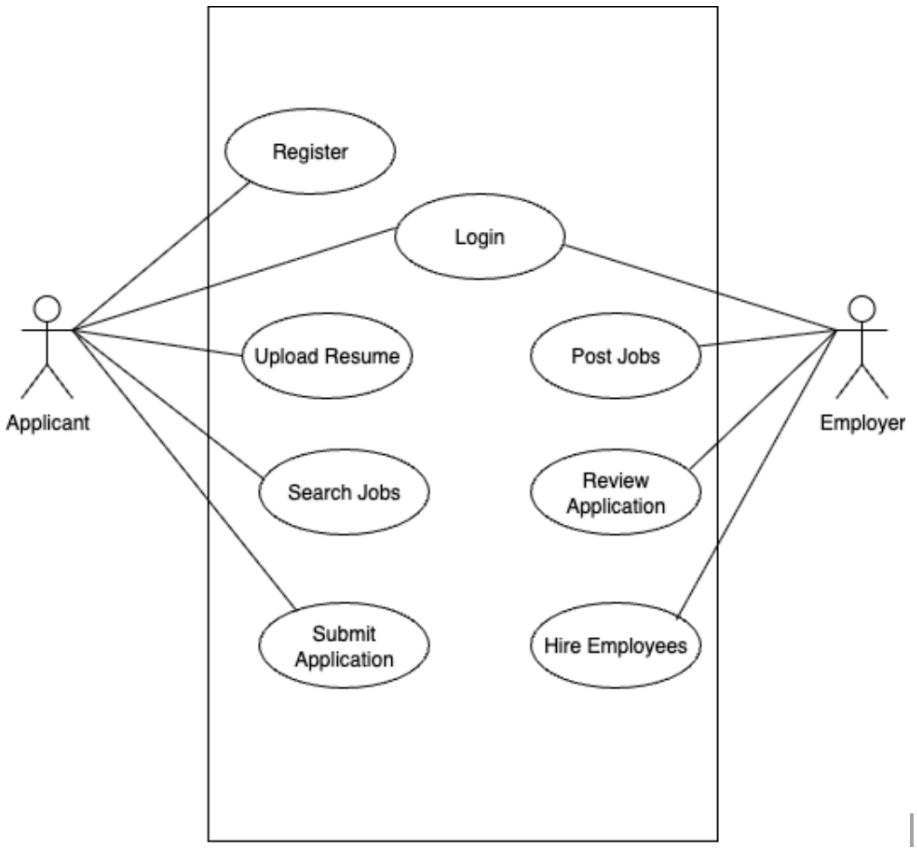


Figure 2: Use Case Diagram

3.1.2 Non-functional Requirements

a) **Scalability:** The system must be able to handle an increasing number of users and job postings without performance degradation.
b) **Security:** User data, including credentials and resumes must be protected with encryption and meet privacy standards.
c) **Usability:** The interface must be intuitive, with a response time of less than 2 seconds, ensuring smooth user experience.
d) **Availability:** The platform must be accessible 24/7, with minimal downtime for maintenance or updates.
e) **Performance:** The cosine similarity algorithm must process matches within a few seconds for a batch of 100s of resumes and job listings.
f) **Compatibility:** The website must be fully functional and responsive on all screen sizes(PC, Tablets, Mobile phones).
g) **Maintainability:** The codebase must be well documented so that it can be read easily by future programmers for easy updates and bug fixes.

3.2 Feasibility Study

The project deals with different fields ranging from system-required technologies, budget and operational factors such as user-friendliness, legality, usability, etc. Conducting a feasibility study was imperative to evaluate the viability of implementing our project. It encompassed the following key aspects:

3.2.1 Technical Feasibility

Technical feasibility assessment focused on the system's compatibility with existing technologies and infrastructure. The study confirmed that the MERN stack, consisting of MongoDB, ExpressJS, ReactJS and NodeJS, was the best suited technology for the project because of its simplicity and scalability.

Official documentation of the related programming languages were taken as reference to learn MERN stack. In addition,a free course was completed in order to sharpen our skills.

Sites:

1. MongoDB Documentation : https://www.mongodb.com/docs/

2. ExpressJS Documentation: https://expressjs.com/
3. ReactJS Documentation: https://react.dev
4. NodeJS Documentation: https://nodejs.org/docs/latest/api/
5. Build a Bookstore using MERN Stack, a freecodecamp course: https://www.freecodecamp.org/news/mern-stack-crash-course/

3.2.2 Operational Feasibility

Operational feasibility focused on how easy the final product will be to operate and maintain after its deployment. Along with this, other scopes determining usability of the product were also studied to make the system operational at all times.

3.2.3 Economic Feasibility

Economic feasibility study dealt with the cost and benefit of the project. A detailed analysis was carried out to estimate the cost of project development, which includes hardware and software resources, design and development cost along with day to day operation cost.

3.3 Analysis

3.3.1 Data Modelling: Entity-Relationship (ER) Diagram

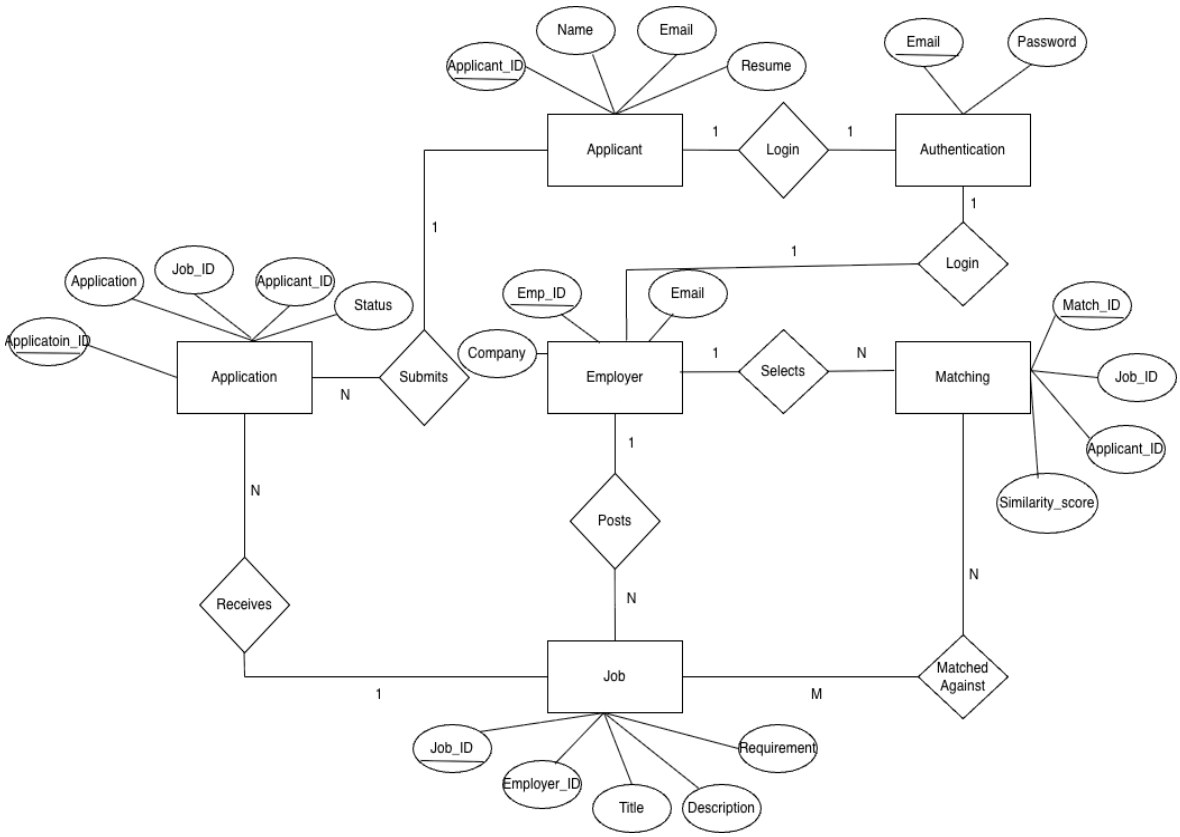


Figure 3: Entity Relationship (ER) Diagram

The above ER-diagram shows entities, attributes and relationships between multiple entities.

Entities:

- **Applicant:** Users who are seeking a job in careerNexus with attributes like Applicant_ID, Name, Email and Resume.
- **Employer:** Users who are searching for potential employees for their organization.
- **Authentication:** A system that verifies the role of the user and gives role based access.
- **Application:** A formal submission made by a job showing interest in a specific job.
- **Job:** A position posted by an employer.

- **Matching:** A process of comparing applicant's resume with job requirement using cosine similarity algorithm

Attributes:

Attributes are the pieces of information that describe an entity, such as an applicant's name, a job's title and an application's status.

Relationships:

Relationships describe how two or more entities are logically connected and interact with each other. They define the way data in one entity is associated with data in another.

- **Applicant-Application Relationship:** An applicant can submit one or more applications.
- **Job-Application Relationship:** A job receives many applications.
- **Employer-Job Relationship:** An employer posts multiple jobs.
- **Application-Matching:** An application is evaluated with jobs in the matching process.
- **Job-Matching:** A job is matched against applicant resumes.
- **Employer-Matching:** An employer selects the best candidates based on similarity score.

Cardinality:

All the relationships follow one-to-many cardinality in the ER-diagram.

- A single applicant can submit many applications, but each application belongs to one applicant.
- A single job can receive multiple applications, but each application is for one job.
- An applicant can be evaluated against many jobs, but each matching record links to one applicant.

3.3.2 Process Modelling: Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) is a visual representation of the flow of data within the system. It helps to understand how the data flows throughout the system and the changes it goes through during processing. It describes the work flow in a graphic format.

a. Context Level DFD (Level 0)

The Level 0 DFD shows the flow of data of the application. It is also called a context diagram. It is a basic overview of the whole system that is being designed or modelled.

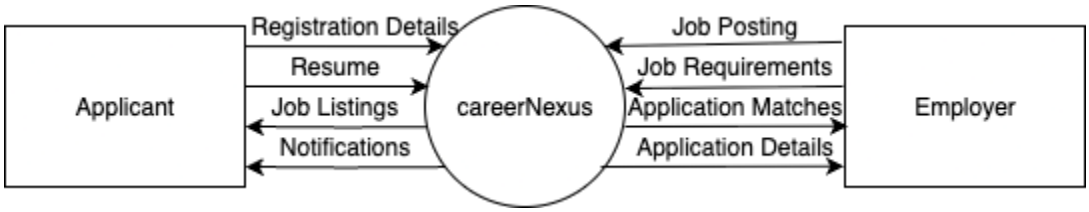


Figure 4: Level 0 DFD

b. Level 1 DFD

Level 1 DFD is the breakdown of the Context Level DFD (Level 0) into separate functionalities of their own. This DFD describes the main functions carried out by the systems, as we break down the high-level process into sub-processes.

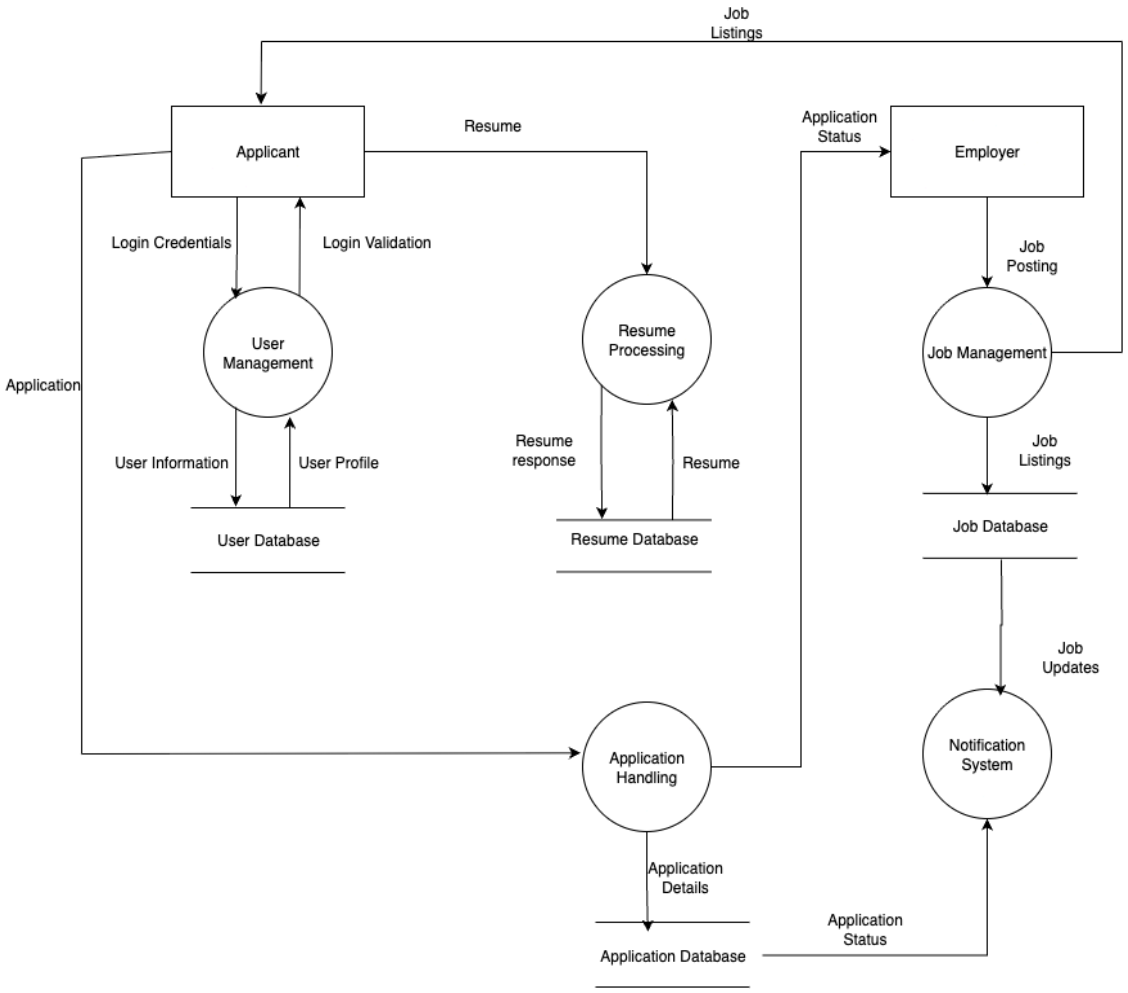


Figure 5: Level 1 DFD

CHAPTER 4:SYSTEM DESIGN

This section defines the basic views of the system. It is about planning how the system will work behind the scenes and how it will look to the users.

4.1 Architectural Design

Architectural design is the process of defining a collection of hardware and software components and their interfaces to establish the framework for the development of a computer system. It involves making high-level decisions on how different components will interact, ensuring scalability and maintainability along with the meeting of project requirements.

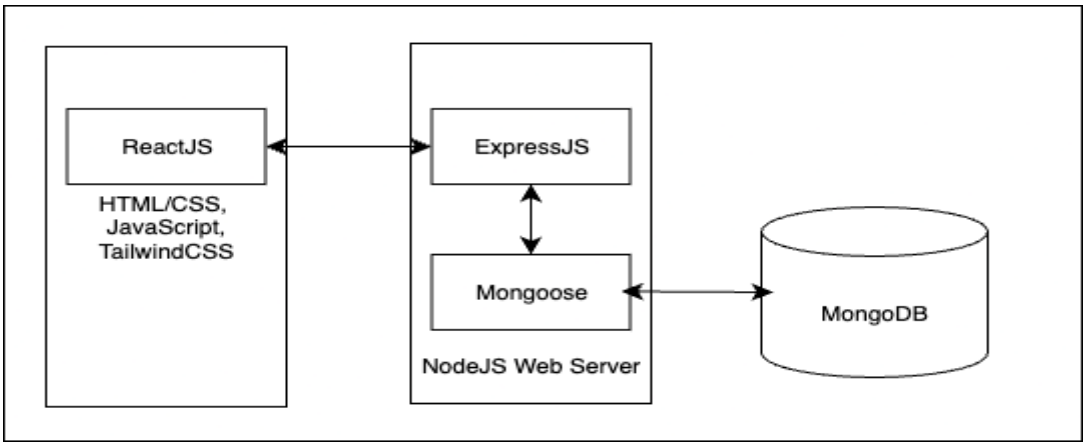


Figure 6: Architectural Design

4.2 Database Schema Design

Database schema design is the process of defining the structure that organizes and represents data in a database system. It involves specifying tables, fields, relationships, and constraints to ensure efficient data storage, retrieval, and integrity.

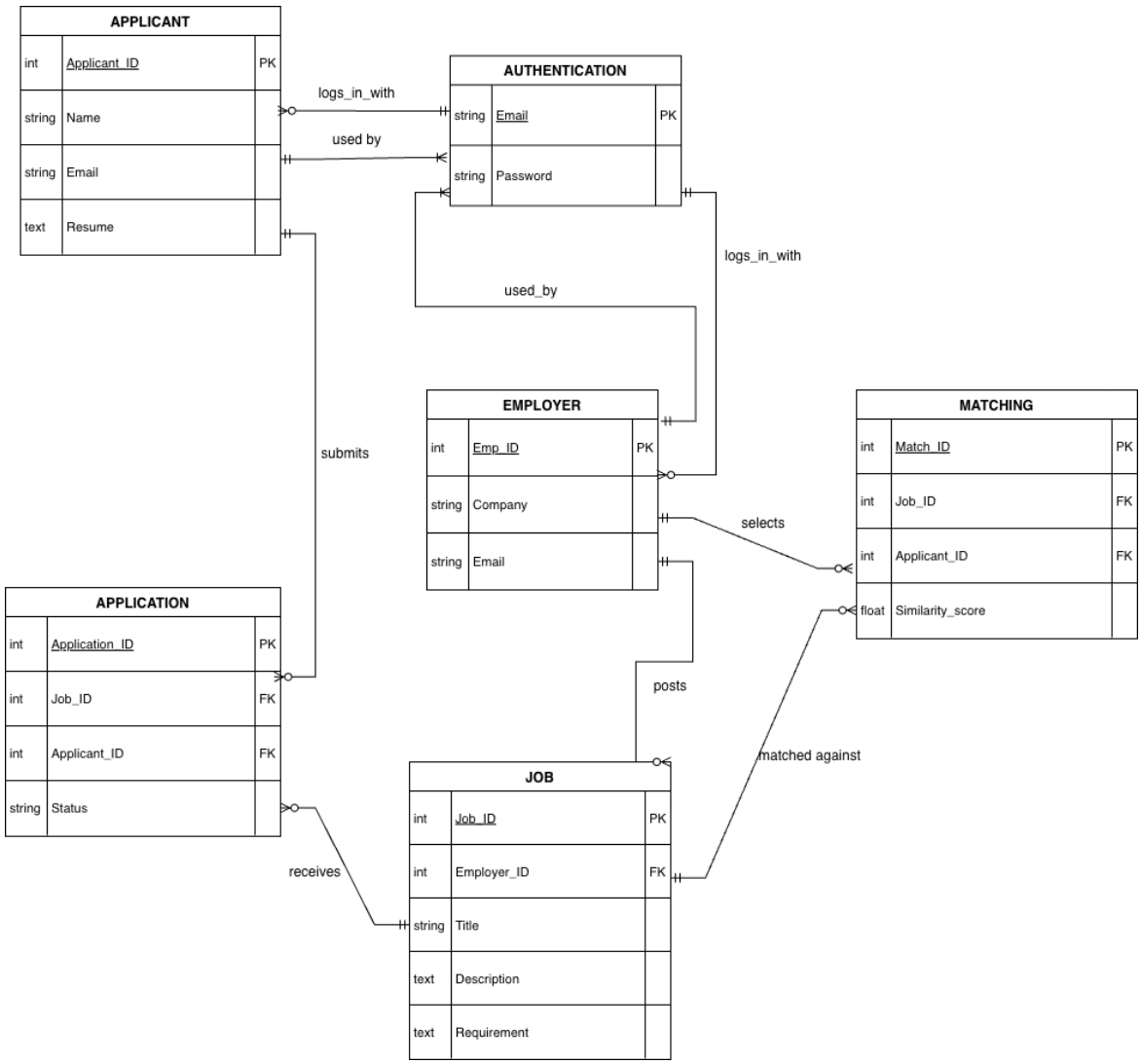


Figure 7: Database Schema Design

4.3 Process Design

4.3.1 Login/ Registration Flowchart

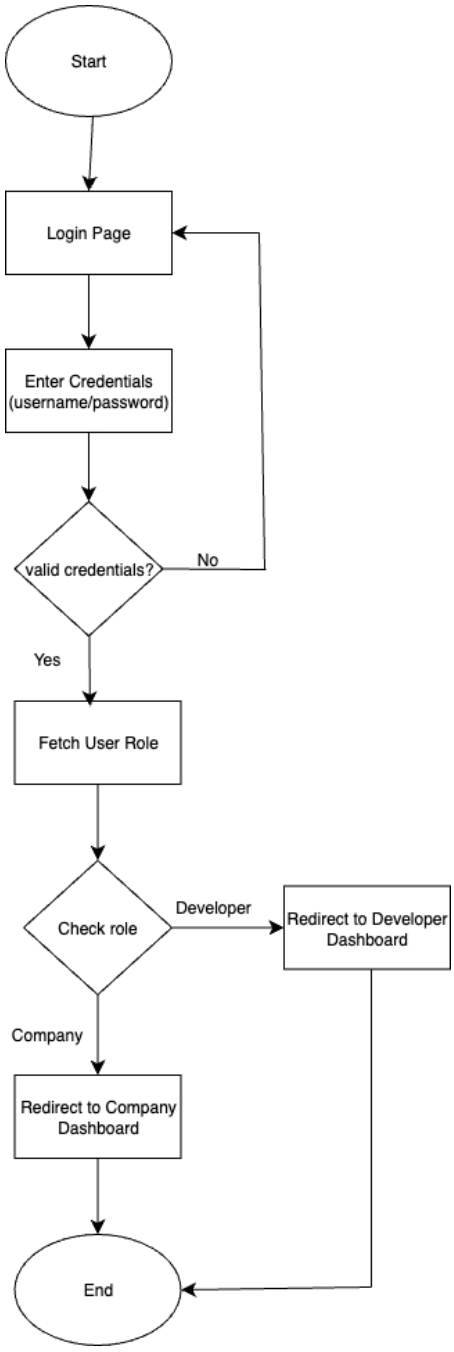


Figure 8: Login/Registration Flowchart

4.3.2 Resume Matching Flowchart

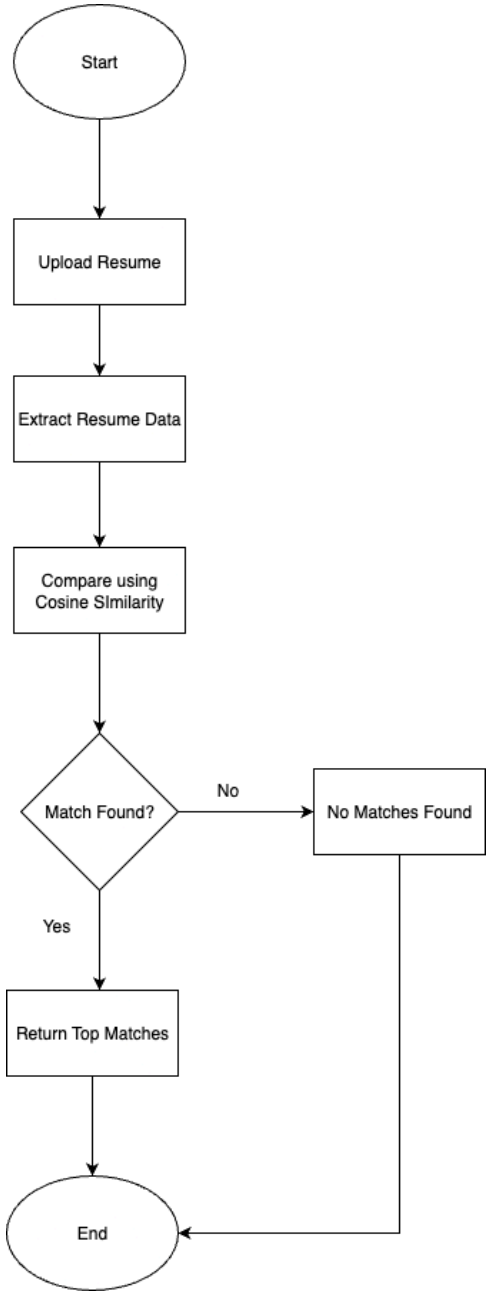


Figure 9: Resume Matching Flowchart

4.3.3 Job Posting Flowchart

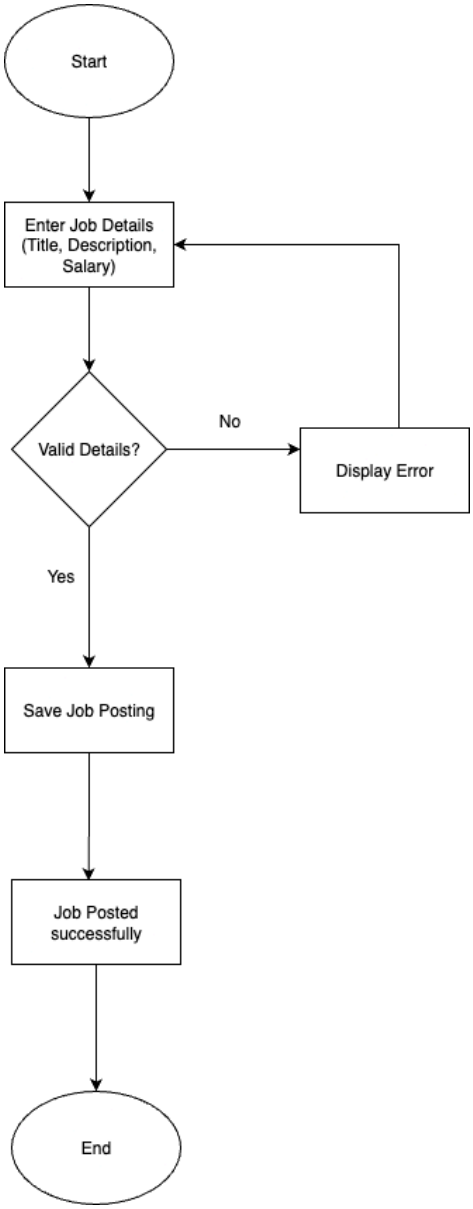


Figure 10: Job PostingFlowchart

4.3.4 Job Application Submission

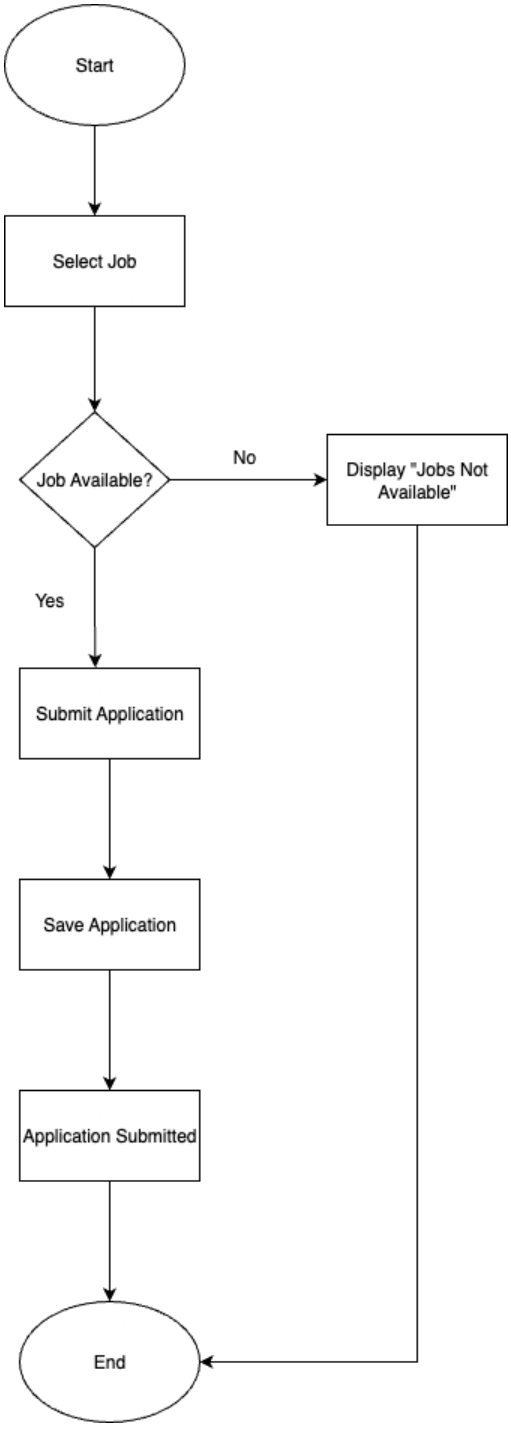


Figure 11:Application Submission Flowchart

4.4 Forms and Interface Design

4.4.1 Form Design

Forms are used to get data from the users, as well as by employers to post jobs.

The different forms required for the system include:

i. In the user's side

- Form for Login and Sign-Up
- Form for updating User Profile
- Form for Social Media feed

ii. In the employer's side

- Form for Login and Sign-Up
- For for posting jobs
- Form for Social Media feed

The forms are created using different elements of the <form>…</form> tag as per requirement and using JavaScript for the Validation purpose.

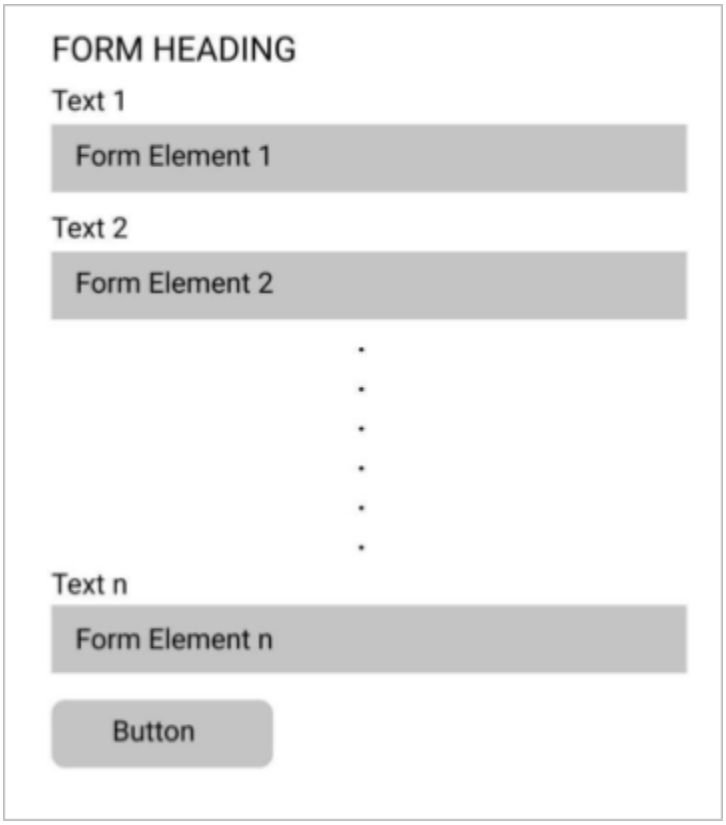


Figure 12: Basic Layout of Forms

4.4.2 Interface Design

Interface Design, often referred to as UI/UX design, involves creating user-friendly and visually appealing interfaces for software or websites. It focuses on enhancing user experience by optimizing layout, navigation, and visual elements, ensuring an intuitive and enjoyable interaction with the product. Good UI/UX design aims to make technology accessible, easy to use, and aesthetically pleasing for users.

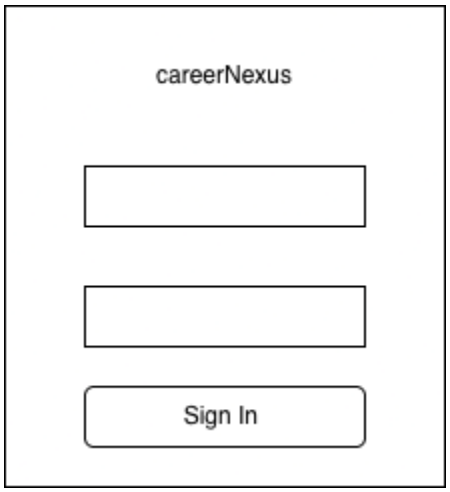


Figure 13: Interface for Sign In Form

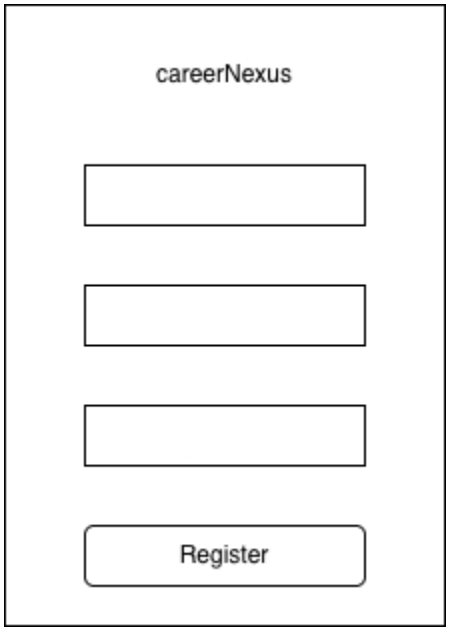


Figure 14: Interface for Sign Up Form

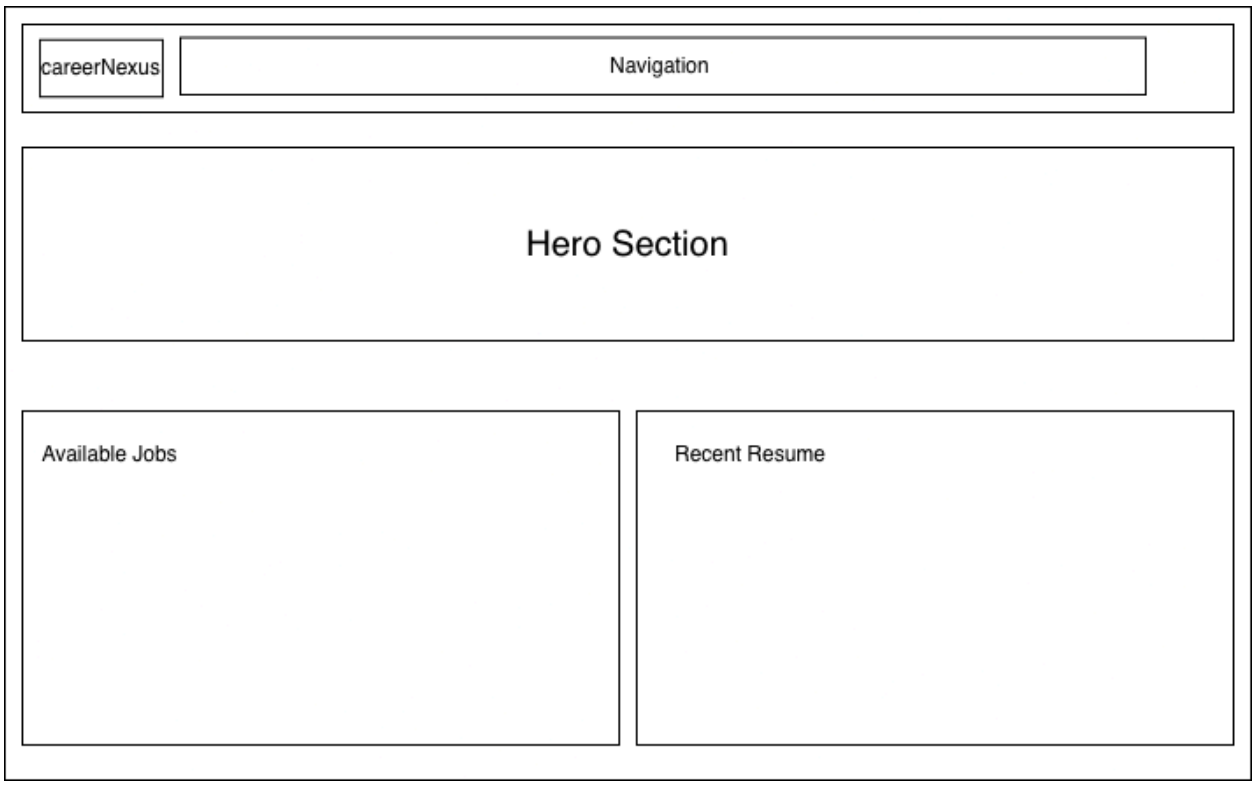


Figure 15: Interface for Dashboard

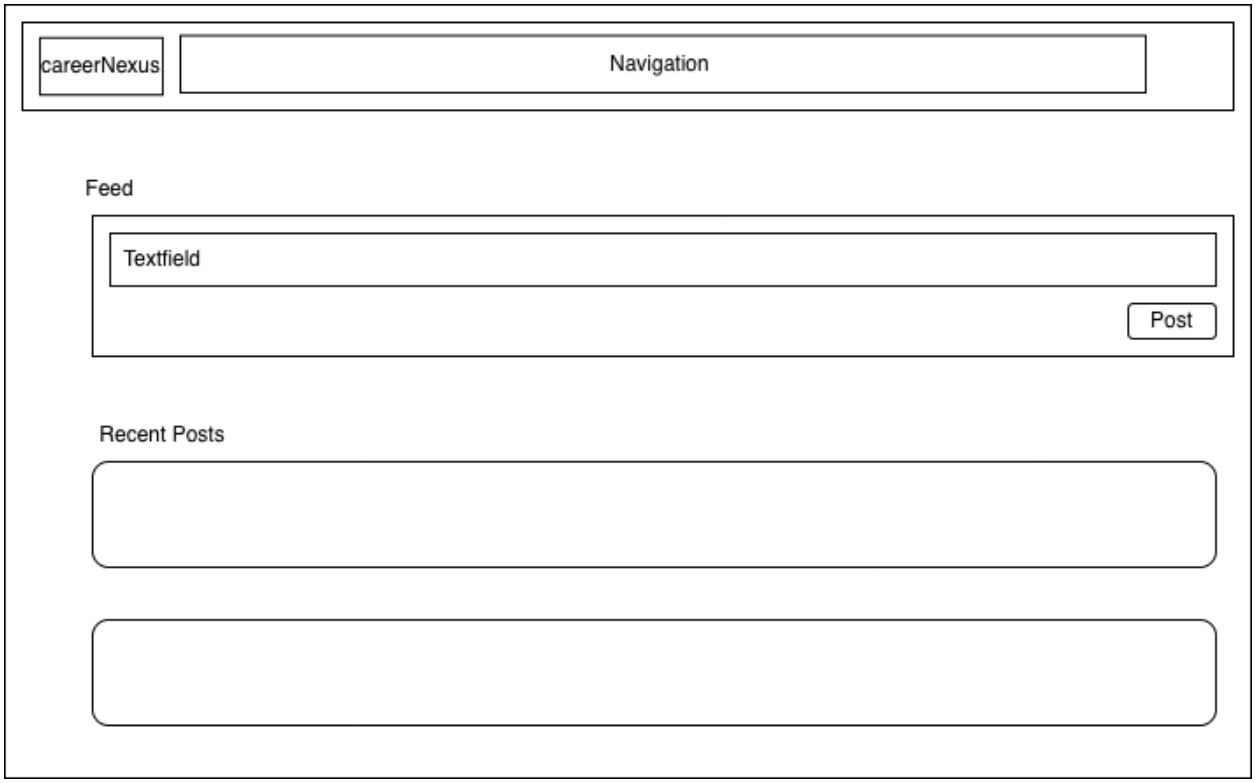


Figure 16: Interface for Social Feed

4.5 Algorithm Details

careerNexus primarily uses the cosine similarity algorithm to enhance the efficiency and accuracy of matching job seekers with suitable job opportunities. It is a cornerstone algorithm that plays a pivotal role in the resume matching process. This section provides a theoretical and mathematical explanation of cosine similarity, detailing its application within the system.

Theoretical Explanation

Cosine similarity is a metric used to measure the similarity between two non-zero vectors in an inner product space. It is particularly effective in text analysis and information retrieval tasks, such as matching resumes to job descriptions, because it focuses on the orientation of vectors rather than their magnitude. In the context of careerNexus, cosine similarity is utilized to compare the vector representations of a job seeker's resume and a job posting. This approach transcends traditional keyword-based matching by capturing the semantic relationship between the content, enabling the system to identify relevant matches even when exact phrases differ.

The algorithm works by treating the text data as vectors in multi-dimensional space. The cosine of the angle between these vectors determines the similarity. Which means that smaller the angle, higher the similarity. A value of 1 represents perfect similarity, 0 represents no similarity whereas -1 represents opposite similarity.

Mathematical Explanation

Cosine similarity is defined mathematically as the dot product of two vectors divided by the product of their magnitudes. For two vectors A and B, the formula is:

$$\text{Cosine Similarity} = \cos(\theta) = \frac{\vec{A}.\vec{B}}{|\vec{A}|.|\vec{B}|}$$

where, $\vec{A}.\vec{B}$ is the dot products of vectors A and B calculated as:

$$\vec{A}.\vec{B} = \sum_{i=1}^{n} A_i \, . \, B_i$$

and $|\vec{A}|$ and $|\vec{B}|$ are the magnitudes of the vectors, calculated as

$$|\vec{A}| = \sqrt{\sum_{i=1}^{n} A_i^2}$$

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Implementation

This is the phase where our project design was actually coded, tested and deployed. We transformed the project blueprint into a functional software using necessary tools and technologies required to do so.

5.1.1 Tools Used

The various system tools that have been used to develop both the frontend and backend of the project are discussed in this section. The project utilizes JavaScript as the primary programming language. The CASE tools used for the project are based on the project requirements. Following case tools have been used for this project:

IDEs (Integrated Development Environments): Visual Studio Code (VSCode is a lightweight and versatile code editor commonly used for JavaScript based development.

Diagramming Tools: Draw.io is used for creating database schema design, Entity-Relationship(ER) diagrams and other visual representations of system architecture.

Table 1: Tools Used in Project

Tools	Purpose
React Js	Frontend Development
Express Js	Backend Development
Node Js	JavaScript Runtime
MongoDB	Database
VS Code IDE	Coding and Development
Postman	API Testing
Figma	Designing wireframe and UI
GitHub	Version Control

1. **ReactJS (Frontend Development) :** ReactJs is a JavaScript library for building Single Page Application (SPA) with smooth User Interface and a component-based architecture, which is efficient for both users as well as programmers.

2. **ExpressJS (Backend Development) :** ExpressJS is a minimal and flexible NodeJS web app framework ideal for building robust and scalable backend services. It simplifies API creation process and server-side logic by seamlessly integrating with frontend to deliver a web application

3. **NodeJS (JavaScript Runtime) :** NodeJS is a JavaScript runtime that enables server side execution of JavaScript. It powers the backend of the application, facilitating efficient handling of requests, data processing and interactions with the database.

4. **MongoDB (Database) :** MongoDB is a NoSQL database platform used for storing and retrieving data. Its flexible and document-oriented structure aligns well with JS based applications, providing scalability and ease of integration with MERN stack.

5. **VS Code (Coding and Development):** Visual Studio Code(VS Code) is a versatile, lightweight integrated development environment (IDE) widely favoured for its rich features, extensions and excellent support for JavaScript and React development. It streamlines coding and enhances developer productivity.

6. **Postman (API Testing) :** Postman is a comprehensive API development and testing tool. It simplifies the testing and validation of API endpoints in the backend.

7. **Figma (UI Design) :** Figma is a collaborative design tool used to create wireframes and designing User Interfaces. It ensures a visually appealing and user-friendly frontend design.

8. **GitHub (Version Control) :** GitHub is a web-based platform for version control and collaboration. It enables developers to track changes, manage code repositories and collaborate on projects.

5.1.2 Implementation Details of Modules

1. Authentication Module

This module handles user registration, securing the login and role based access control for job seekers, employers and admins.

- Passwords are hashed before storing in MongoDB.
- JWT Tokens are issued after successful login.
- Tokens are validated on each request via middleware.
- Only authorized users have access to specific features.

2. Frontend Module

This module is built with ReactJS and provides dynamic user interfaces as per the role of the user. It uses Axios for API communication and Context API for state management.

- Job seekers can upload resumes, view jobs and apply filters to the jobs available.
- Employers can post jobs, view applicants and track applications.
- Admin dashboard displays analytics, user activity and job listing tools.

3. Backend Module

This module is developed using NodeJS, ExpressJS and MongoDB. It is responsible to manage all server-side logic, API routing and business logics. It implements RESTful endpoints with proper validation and error handling.

- CRUD operations for users, jobs, applications and admin logs.
- Role specific logic enforced at the controller level.
- Global error handler returns consistent JSON responses.

4. Resume Matching Module

It implements job recommendation using the cosine similarity between resume and job description vectors. It processes text data to extract keywords for accurate matching.

- Uses TF-IDF vectorization and cosine similarity algorithm for scoring.
- Returns match percentage to rank and recommend jobs dynamically.
- Integrated into the job search and dashboard section for real-time personalization.

5. Middleware and Access Control

It secures routes and enforces access restrictions based on user roles and authentication status.

- JWT verification middleware protects private routes.
- Role-based guards restrict access.
- Prevents unauthorized access to admin and user-specific endpoints.
- Handles token expiration and invalid signature errors.

6. API and Routing Structure

It organises all endpoints following RESTful conventions with modular routing.

- Routes are structured as per their functionality.
- Uses express.Router() for modularity and reusability.
- Applies authentication and role checking middleware at routing layer.
- Ensures secure, scalable and maintainable API design

5.2 Testing

Based on the software requirement specification, testing was performed to investigate and validate the behavior of a fully integrated software product. Before deploying an application or website, it must be thoroughly tested. As a result, this application was tested. Some of the types of testing that we did are described below.

5.2.1 Unit Testing

A unit test is a way of testing a unit - the smallest piece of code that can be logically isolated in a system. In most programming languages, that is a function, a subroutine, a method or property.

Table 2 : Test Case for User Registration

SN	Input	Prompt	Expected Output	Actual Output	Result
1	Username: abc	Username length should be at least 5.	Registration Complete	Registration Failed	Fail

	Email: xyz.com Phone: 12345 Password:abc123	Email must have @ symbol. Phone numbers must start from '9' and 10 digits long.Password must have at least one Capital Letter, Number, Unique Symbol			
2	Username: Manju Email: manish758@gmail.com Phone: 9862067366 Password: manish@758	Verify Email, OTP Code Sent to Email	Registration Complete	Registration Successful	Pass

Table 3 : Test Case for User Login

SN	Input	Prompt	Expected Output	Actual Output	Result
1	Email: manish@gmail.com Password: Aty123	Invalid Username or Password	Login Success	Login Failed	Fail
2	Email: manish758@gmail.com Password: manish@758	Logged in Successfully	Login Success	Login Success	Pass

Table 4 : Test Case for Job Posting

SN	Input	Prompt	Expected Output	Actual Output	Result
1	Title:Software Developer Desc: Salary:50000	Title and description must be non-empty. Salary must be numeric.	Job Posting Complete	Job Posting Failed	Fail
2	Title: Data Analyst Desc: Data analysis role Salary: 60000	Validate job details and post successfully.	Job Posting Complete	Job Posting Successful	Pass

Table 5 : Test Case for Job Searching

SN	Input	Prompt	Expected Output	Actual Output	Result
1	Keyword: Python Location: Kathmandu	Search Success with filters.	Searched Jobs Listing	Searched Jobs Listing	Pass
2	Keyword: Python Location: None	Validate location and search.	No Results Found	No Results Found	Pass

Table 6: Test Case For Application Submission

SN	Input	Prompt	Expected Output	Actual Output	Result
1	Job ID: 101 User ID: 001 Resume: None	Resume must be attached	Application Failed	Application Failed	Pass
2	Job ID: 102 User ID: 002 Resume: resume.pdf	Validate location and search.	Application Complete	Application Successful	Pass

Table 7: Test Case For Employer Dashboard

SN	Input	Prompt	Expected Output	Actual Output	Result
1	Filter: New Applications Limit: 5	Display latest applications within limit.	5 Applications Listed	3 Applications Listed	Fail
2	Filter: All Applications Limit: 10	Display all applications within the limit.	10 Applications Listed	10 Applications Listed	Pass

5.2.2 System Testing

System Testing plays a crucial role in ensuring that the entire system functions as intended and meets the specified requirements. System testing is done to check whether the software or product meets the specified requirements or not.

Table 8: System Testing

SN	Component	Test Objective	Test Inputs	Expected Output	Actual Output	Result
1	User Authentication	Verify secure login and session management across all user types.	Valid/Invalid email, password, token.	Successful login/logout, error messages.	Successful login/logout, error messages.	Pass
2	Job Posting Module	Ensure accurate job creation and	Title, description, salary (valid/invalid)	Job posted with correct details.	Job posted with correct details.	Pass

		visibility to users.				
3	Resume Upload	Validate file upload and integration with matching system.	PDF/DOCX files, invalid formats.	Successful upload, error for invalid files.	Successful upload, error for invalid files.	Pass
4	Job Searching	Test search accuracy with filters and semantic matching.	Keywords, location, category filters.	Relevant job listings returned.	Relevant job listings returned.	Pass
5	Matching Algorithm	Confirm cosine similarity matches resumes to jobs accurately.	Resume text, job description vectors.	Top 5 relevant matches listed.	Top 5 relevant matches listed.	Pass
6	Application Submission	Ensure successful application submission and tracking.	Job ID, User ID, resume reference.	Application recorded in database.	Application recorded in database.	Pass
7	Employer Dashboard	Validate dashboard functionality for managing applications.	Filter criteria, application list.	Updated shortlist and status updates.	Updated shortlist and status updates.	Pass
8	Notification System	Test delivery	User email,	Email sent with	Email sent with	Pass

		and content accuracy of notifications.	notification type.	correct details.	correct details.	
9	Admin Management	Ensure admin control over users, jobs, and system settings.	User list, job approval request.	Approved/ rejected actions logged.	Approved/ rejected actions logged.	Pass

5.3 Result Analysis

The testing and evaluation of careerNexus showed that the system works well and meets its main goals. After running unit tests, system tests, and getting feedback from users, we found that the platform is reliable, easy to use, and effective in matching job seekers with employers.

5.3.1 Login and Registration

The login and registration system worked without any bugs. Users could sign up with a valid email and password. The system checked for already-used emails and made sure passwords had at least 8 characters, including a number and a special symbol.

- **Total Test Cases**: 50
- **Passed**: 48 (96%)
- **Failed**: 2 (due to network timeout, not system error)
- **After Fix**: 100% success

5.3.2 Resume Upload and AI Matching

Users uploaded resumes in PDF or DOCX format. The system read the content and compared it with job descriptions using cosine similarity.

We took a sample resume and job description to show how the matching works.

Sample Resume Text: "*Python, React, SQL, 2 years experience, B.Sc.CSIT"*
Sample Job Description: *"Looking for a developer with Python, React, database skills"*

Converting the data to vectors using TF-IDF,

Table 9: TF-IDF Vector Weights for Cosine Similarity Calculation

Term	**Resume Weight (A)**	**Job Weight(B)**
python	0.6	0.7
react	0.5	0.6
sql	0.4	0.3
experience	0.3	0.0
database	0.0	0.5

Step 1: Dot Product

$A \cdot B = (0.6 \times 0.7) + (0.5 \times 0.6) + (0.4 \times 0.3) + (0.3 \times 0.0) + (0.0 \times 0.5) = 0.84$

Step 2: Magnitude of Vectors

$$|\vec{A}| = \sqrt{(0.6)^2 + (0.5)^2 + (0.4)^2 + (0.3)^2 + (0.0)^2} = \sqrt{0.86} = 0.927$$

$$|\vec{B}| = \sqrt{(0.7)^2 + (0.6)^2 + (0.3)^2 + (0.0)^2 + (0.5)^2} = \sqrt{1.19} = 1.091$$

Step 3: Cosine Similarity

$$\text{Similarity} = \frac{A \cdot B}{|\vec{A}||\vec{B}|} = \frac{0.84}{0.927 \times 1.091} = \frac{0.84}{1.011} = 0.83 \text{ (83\% match)}$$

Overall Matching Results:

- **Total Test Resumes**: 30
- **Correctly Matched**: 26

- **Accuracy**: 86.7%
- **Reason for 4 Mismatches**: Job used different terms (e.g., “frontend” instead of “React”).

5.3.3 Job Posting and Searching

Employers were able to post jobs quickly. All the required fields were checked before saving.

- **Posting Time**: Less than 2 seconds
- **Search Speed**: Results appeared in 0.8 seconds on average
- **Relevance**: 91% of search results matched user intent

The results clearly show that careerNexus works well in real use. The cosine similarity algorithm successfully matches resumes and jobs with 83% similarity in most cases, which is much better than keyword-only systems. With small improvements like adding word synonyms and faster mobile loading, the system can reach 90%+ accuracy. Overall, careerNexus is a reliable, user-friendly, and smart job portal ready for launch.

CHAPTER 6 : CONCLUSION AND FUTURE RECOMMENDATIONS

6.1 Conclusion

careerNexus is an AI driven website, designed with the aim of addressing the evolving demands of the modern recruitment scenario. By using the cosine similarity algorithm, the system delivers precise and efficient matching between resumes and job openings, overcoming the limitations of the traditional keyword-based searching methods, that were mostly miss contextual fit. It ensures a user friendly experience which aligns with the trends observed in successful platforms like LinkedIn. The job listing process is also made easier and the potential employers can browse without any hassles. The system was built using MongoDB, ExpressJS, ReactJS and NodeJs (MERN), making it efficient and scalable. The plan was to make such a setup that could handle the load while keeping everything fast and reliable, be it job seekers uploading resumes or employers browsing candidates.

Moreover, this Project helped us gain a significant technical insight into the intricacies of our chosen technologies such as ReactJS, NodeJS, etc, navigating challenges and finding innovative solutions. Project management taught the value of realistic timelines and adaptive planning. The testing phase underscored the need for robust quality assurance practices and early bug detection. User experience considerations became pivotal, emphasizing iterative design processes and responsive interfaces. Scalability and future enhancements were on our radar, prompting us to build with flexibility in mind. Adaptability was key, as we navigated unforeseen challenges with agility. Knowledge transfer within the team emerged as a priority, ensuring that insights and best practices are documented for future reference. Embracing a culture of continuous improvement, we conducted retrospectives to refine our processes. Effective stakeholder communication proved crucial, underscoring the importance of regular updates and proactive expectation management.

6.2 Future Recommendations

In the future, several enhancements can be made to further improve the system and its capabilities. Capabilities of AI can be expanded by exploring features like current job trends in the market. Moreover, we can add video interviews in the application, which is the next part of hiring after resume scanning. We can add a user feedback section where both kinds of users can give their valuable suggestions regarding the application. Additionally, we can adopt a freemium model of operation, where certain features are unlocked by paying a certain amount of money, which would help in the monetization of our application.

References:

[1] S. Jawale, S. Nehete, H. Patil, S. Pathak, P. Sapale, and S. Zite, “Cosine similarity: A key driver for enhanced recommendation systems,” International Research Journal of Modernization in Engineering Technology and Science, vol. 6, no. 4, pp. 1466–1470, Apr. 2024.

[2] S. Yadav, K. Oza, K. Shah, and S. Patil, “AI-based job portal,” International Journal of Creative Research Thoughts (IJCRT), vol. 12, no. 4, pp. j429–j434, Apr. 2024.

[3] U. Dudwadkar, S. Memane, P. Yadav, A. Ansari, and K. Sakure, “A survey on AI-powered job portal,” International Research Journal of Modernization in Engineering Technology and Science, vol. 7, no. 3, pp. 8059–8066, Mar. 2025.

[4] K. H. Guo, “Testing and validating the cosine similarity measure for textual analysis,” SSRN Electronic Journal, 2023. [Online]. Available: https://ssrn.com/abstract=4258463.

APPENDICES

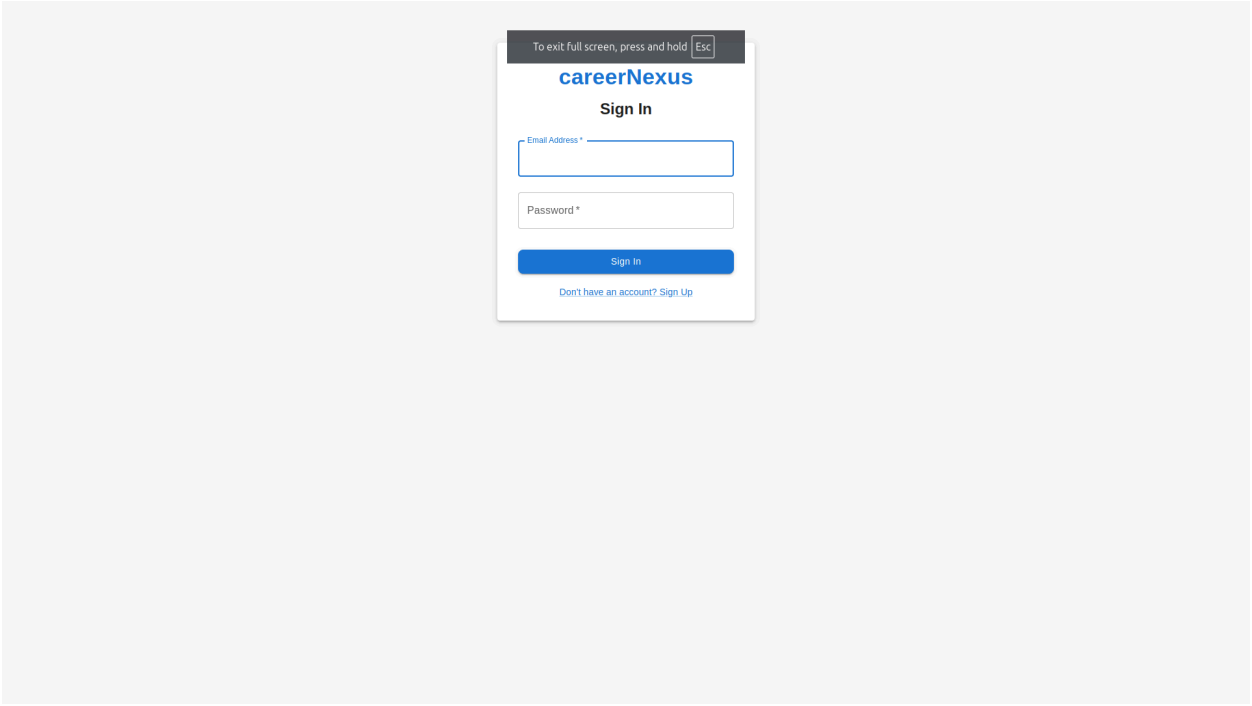


Figure 17: Sign In Form

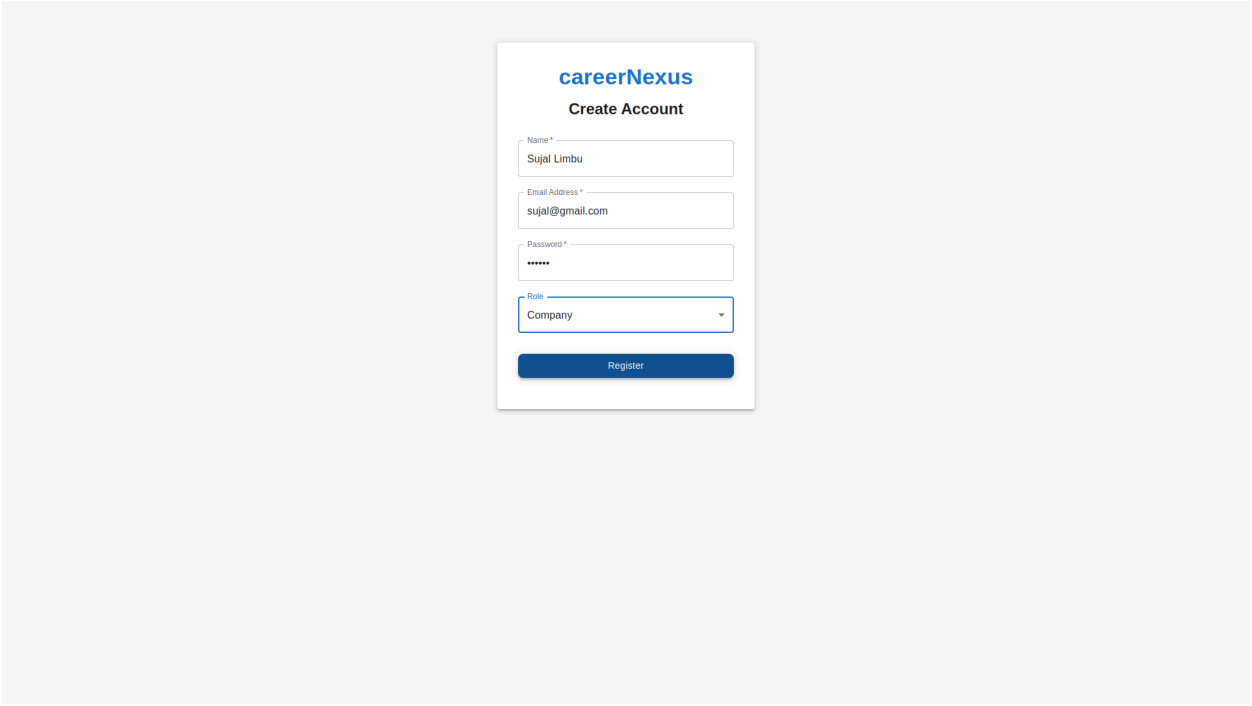


Figure 18: Sign Up form

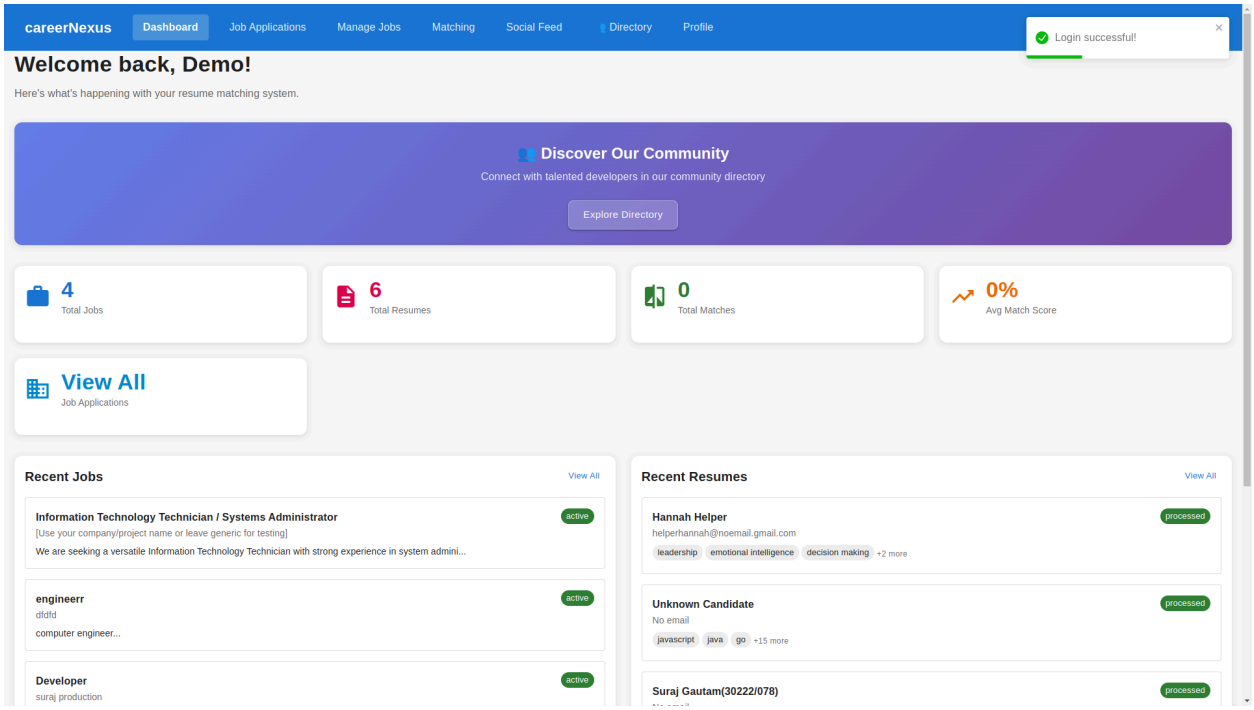


Figure 19:Employer Dashboard

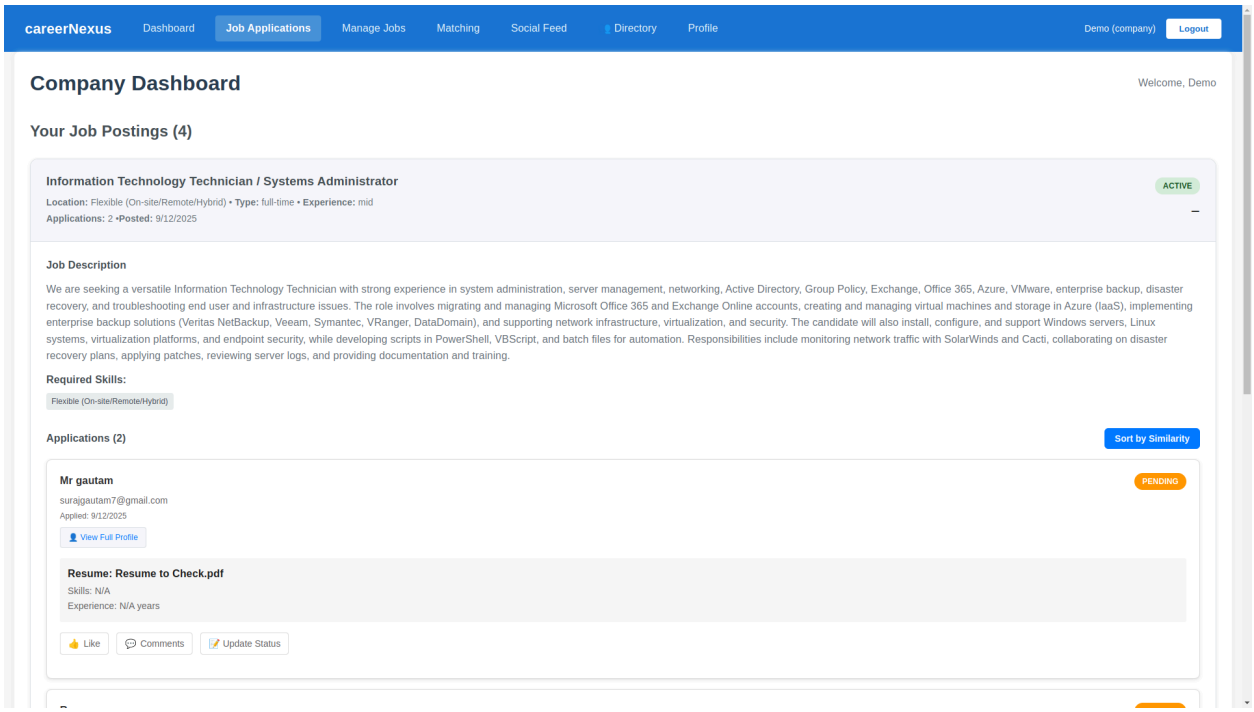


Figure 20: Company Dashboard

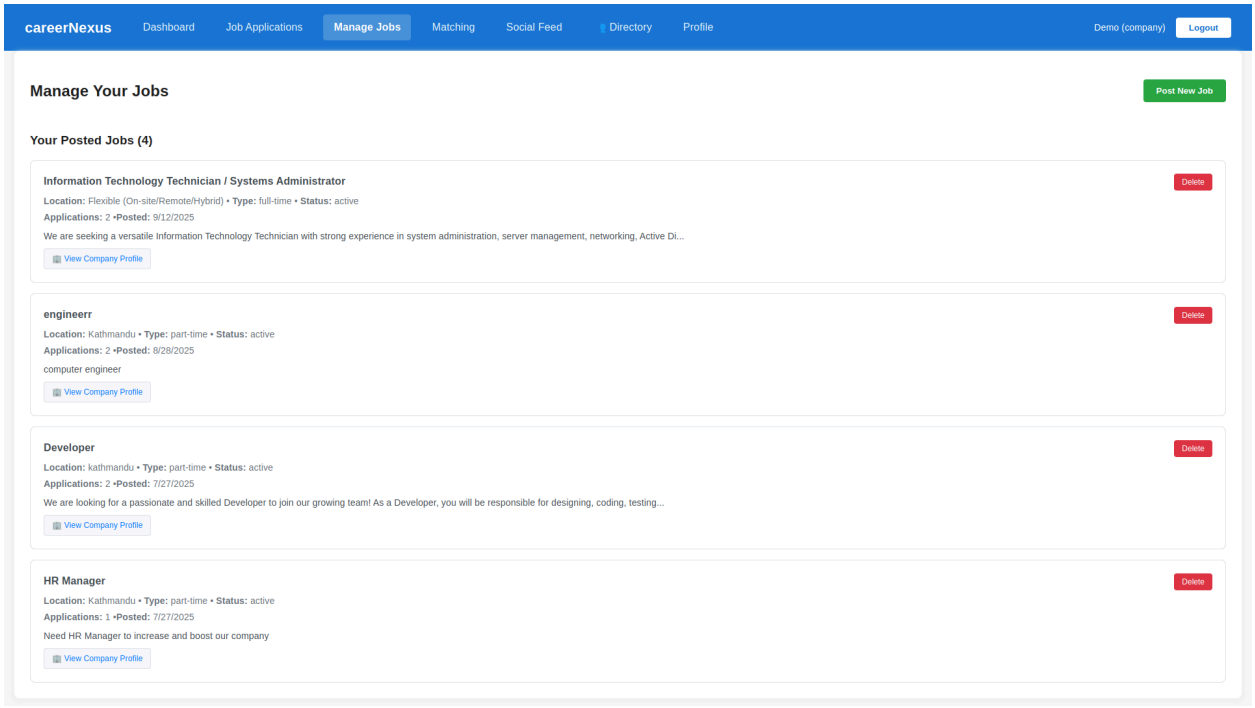


Figure 21: Manage Jobs section

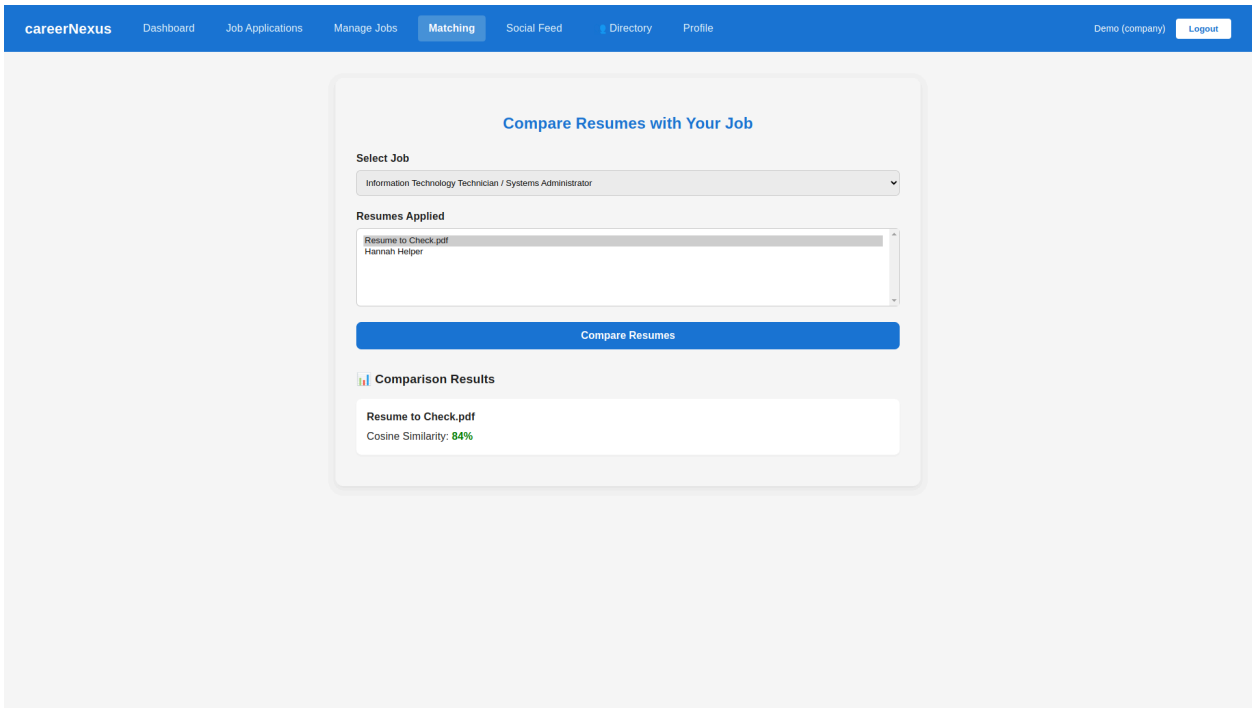


Figure 22: Resume Matching

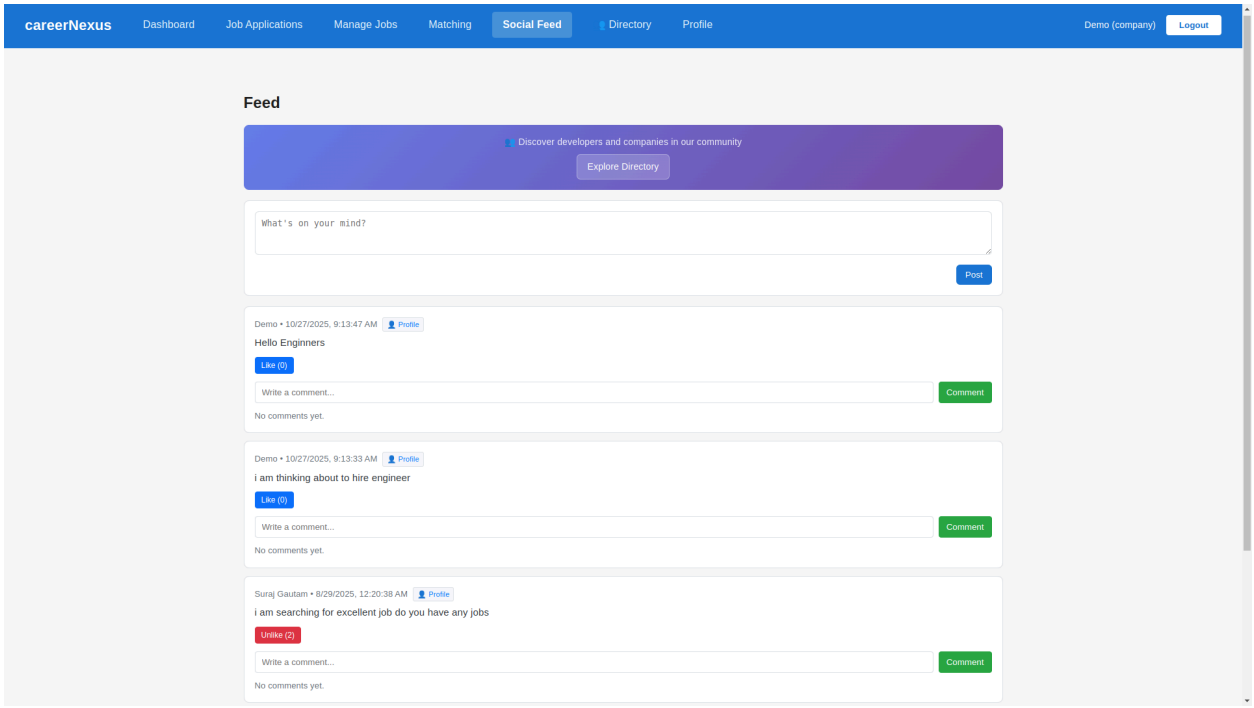


Figure 23: Social Feed

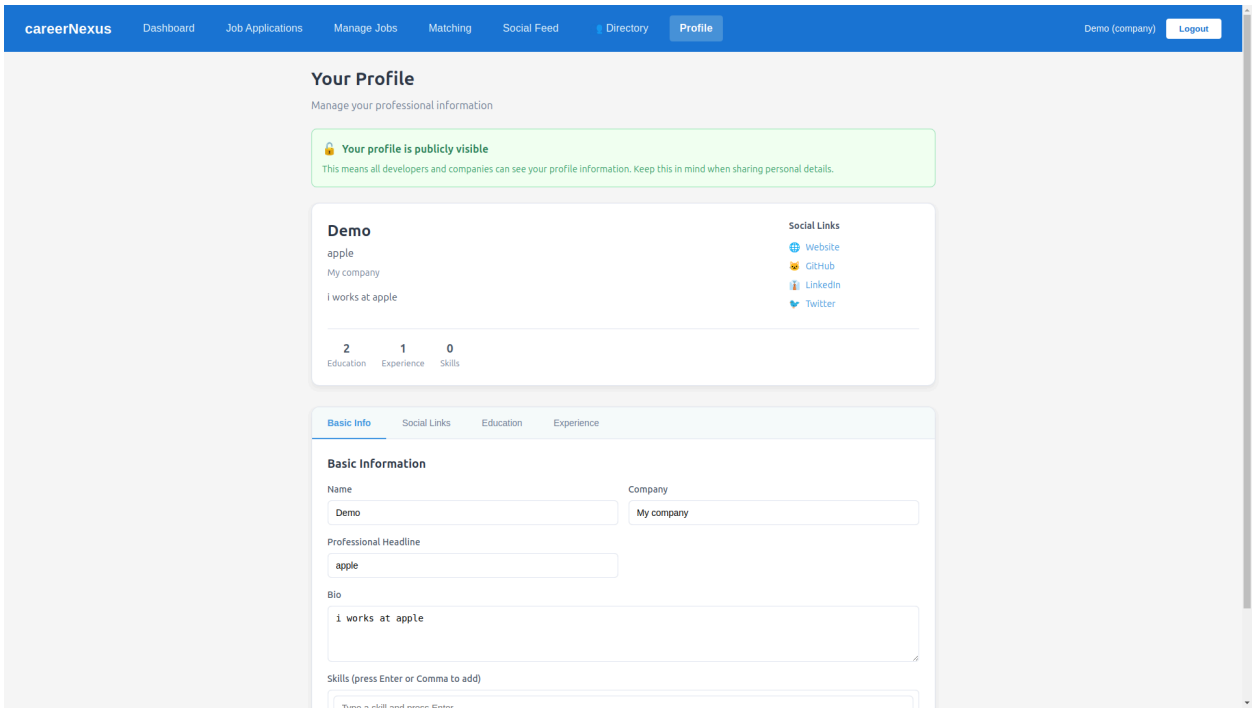


Figure 24: Company Profile

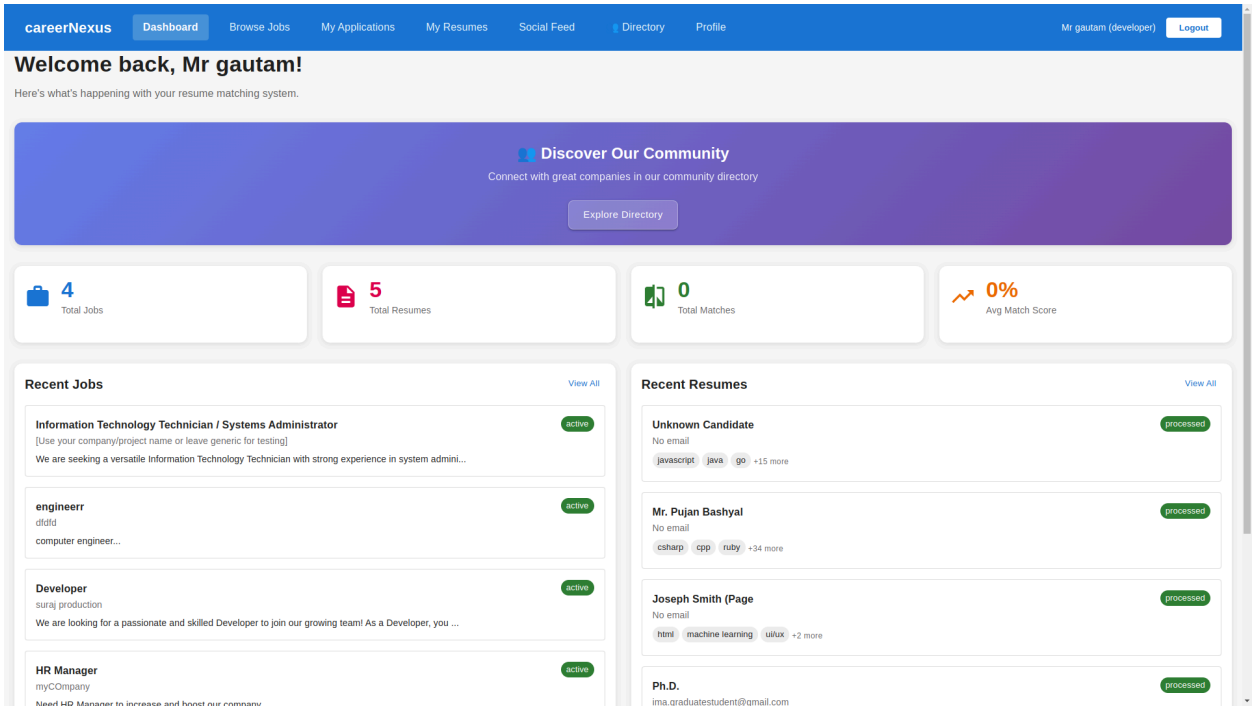


Figure 25: Employee Dashboard

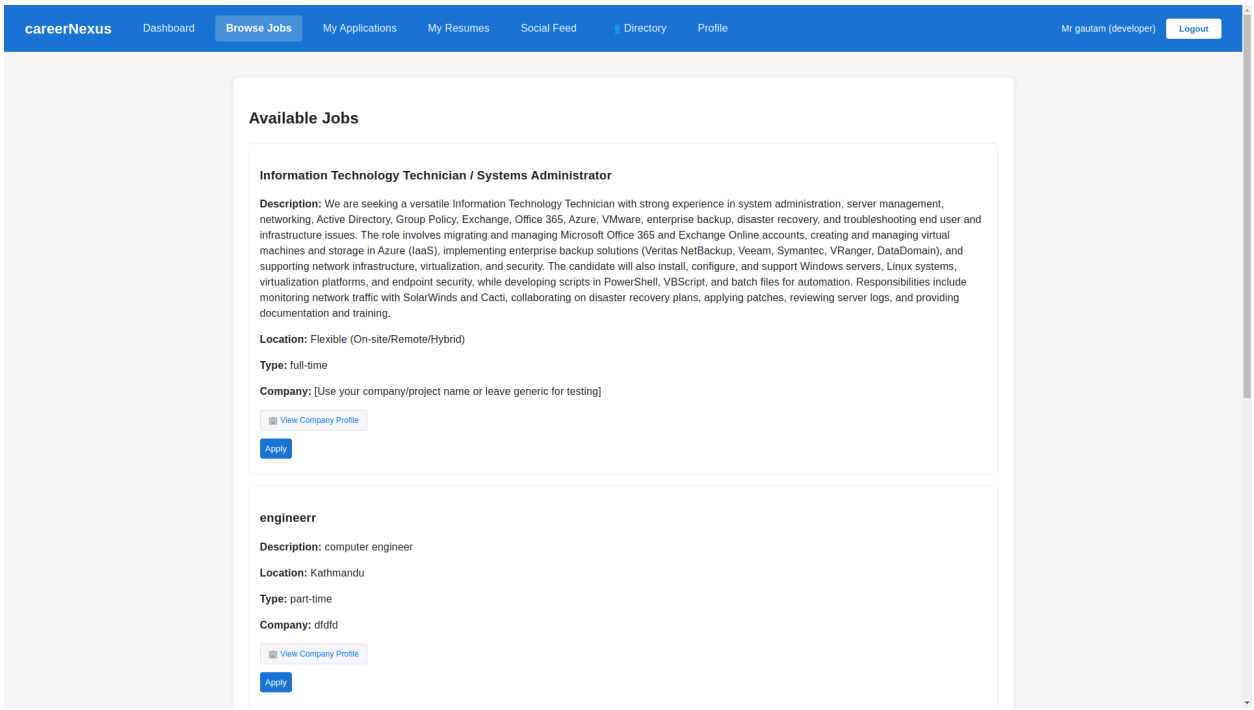


Figure 26: Job Browsing

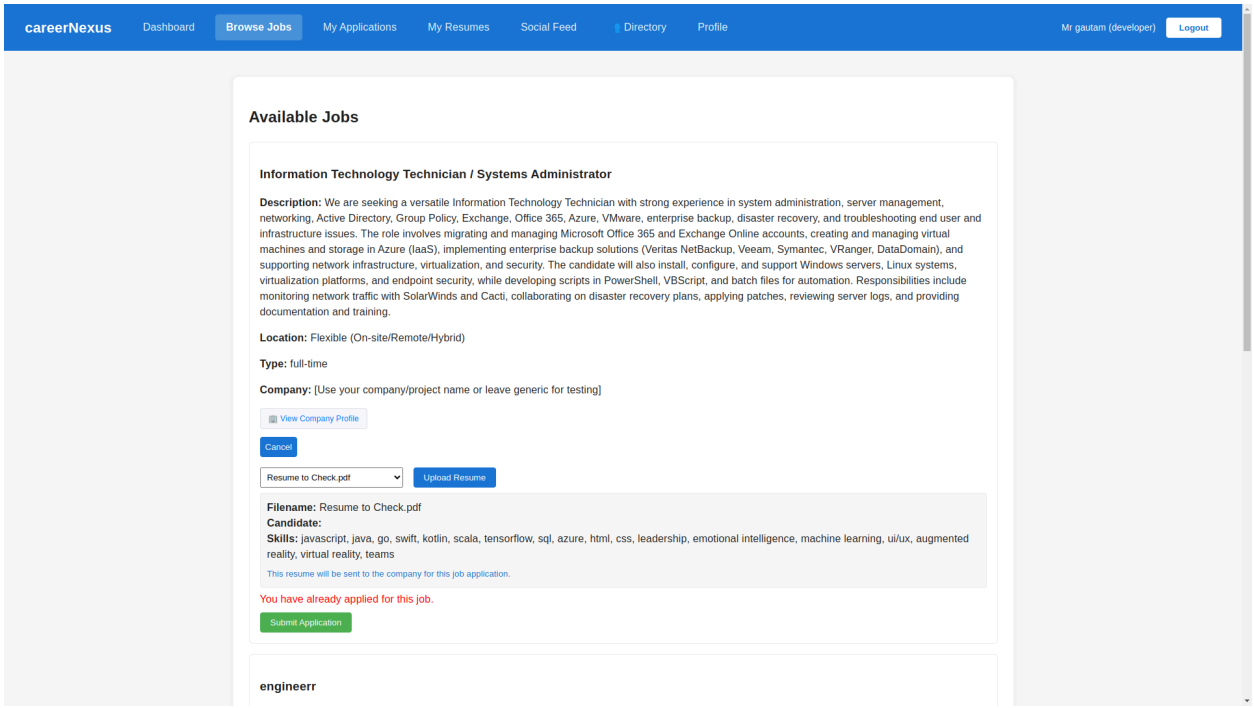


Figure 27: Applying to a Job

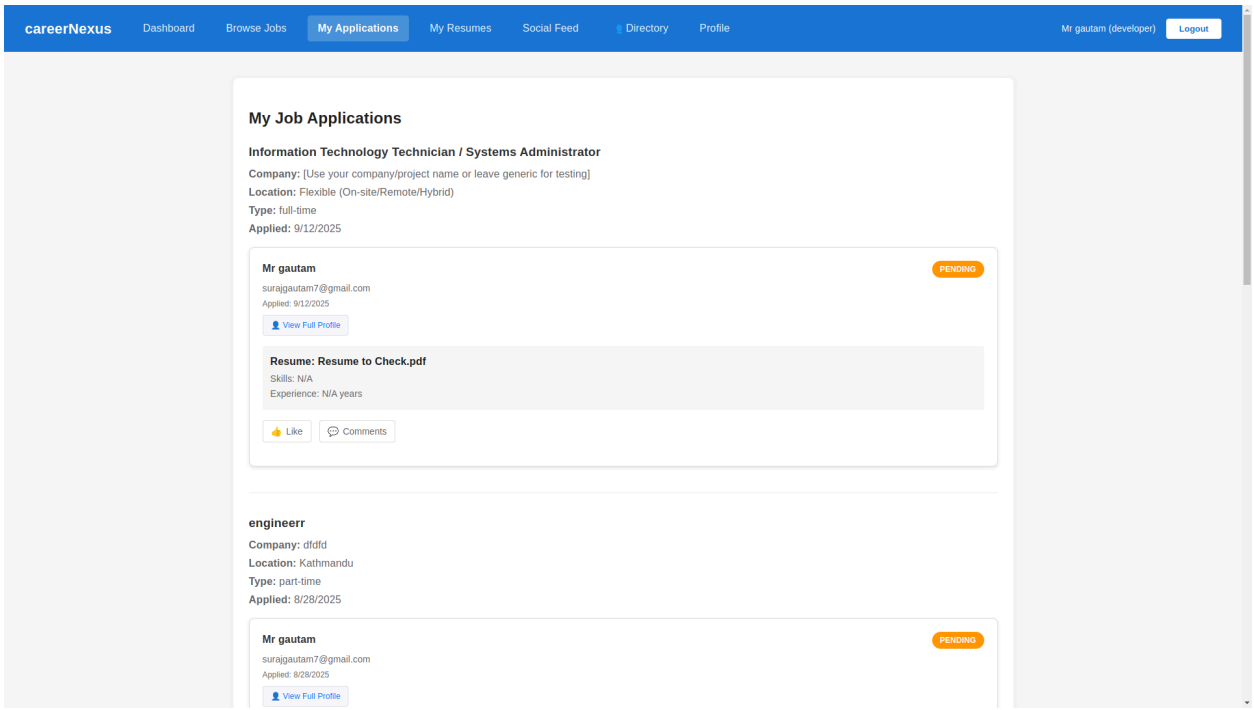


Figure 28: Jobs Applied by a User

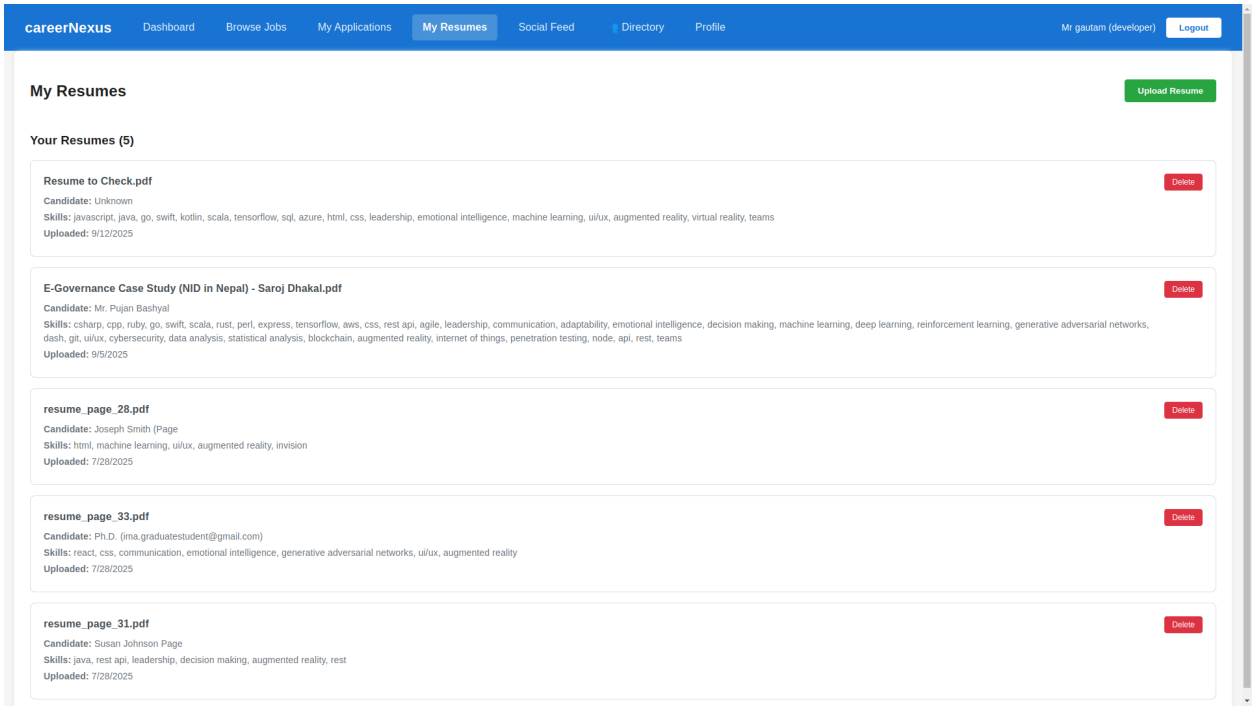


Figure 29: User Resumes

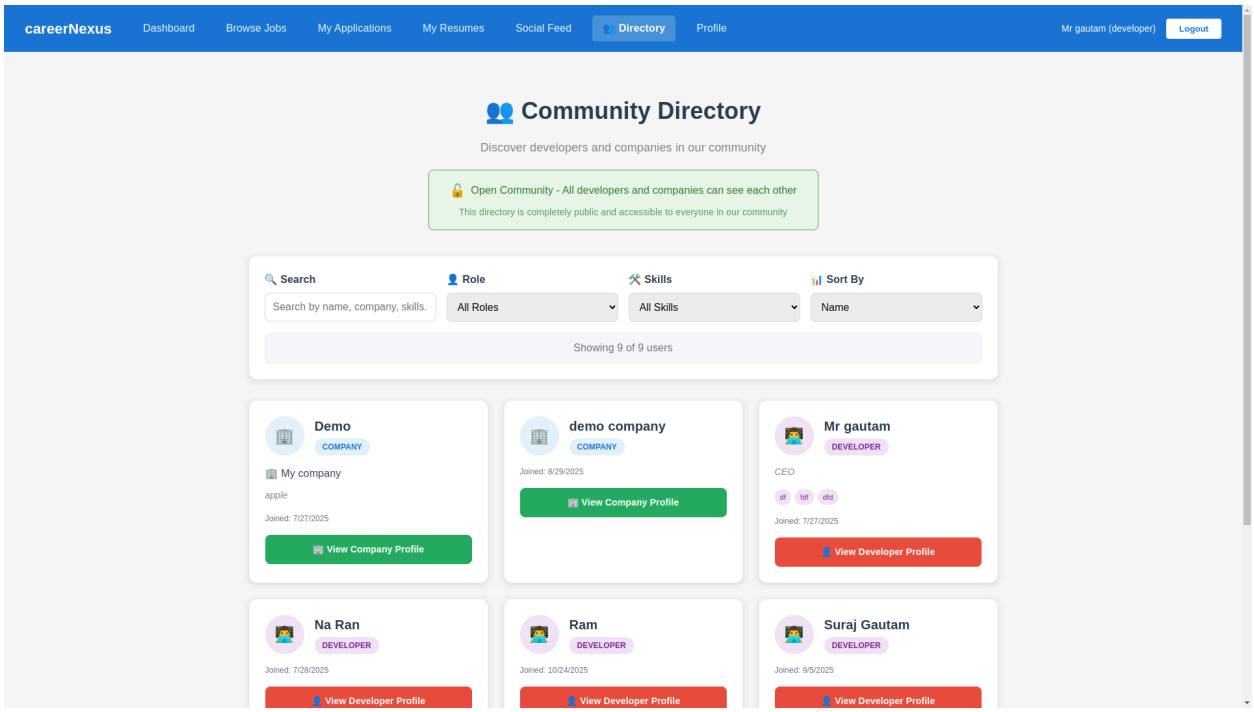


Figure 30: Community Directory

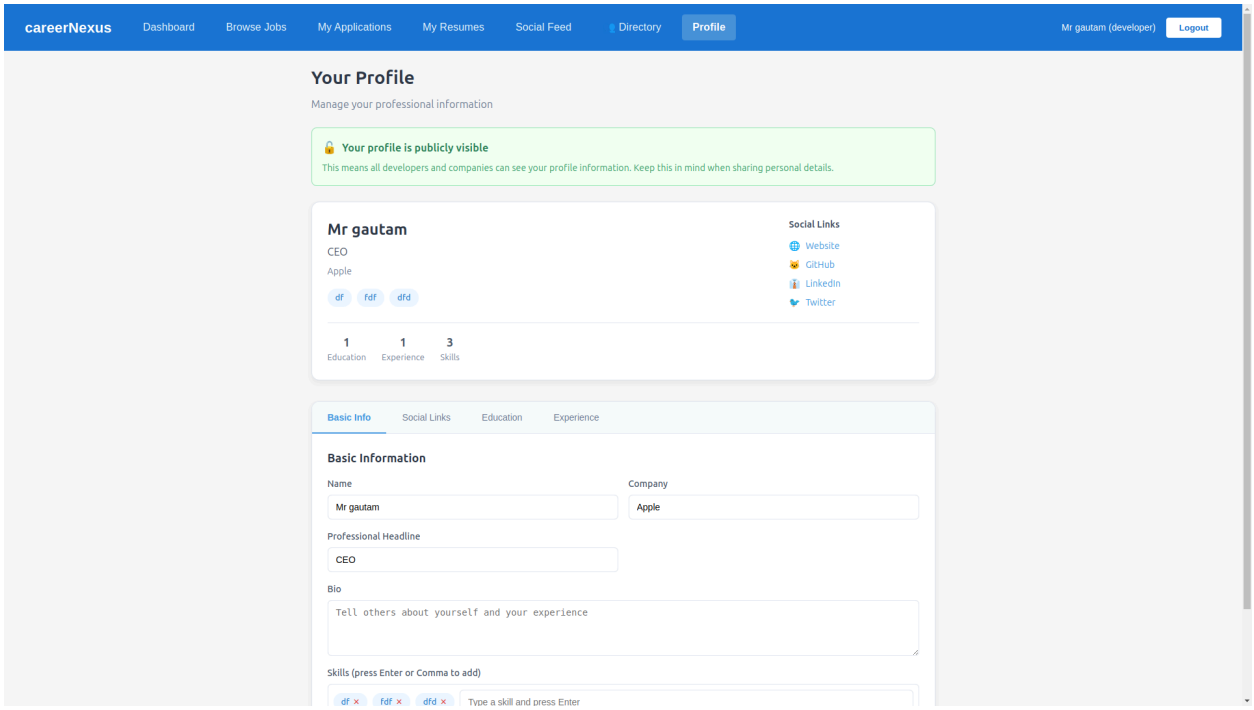


Figure 31: User Profile

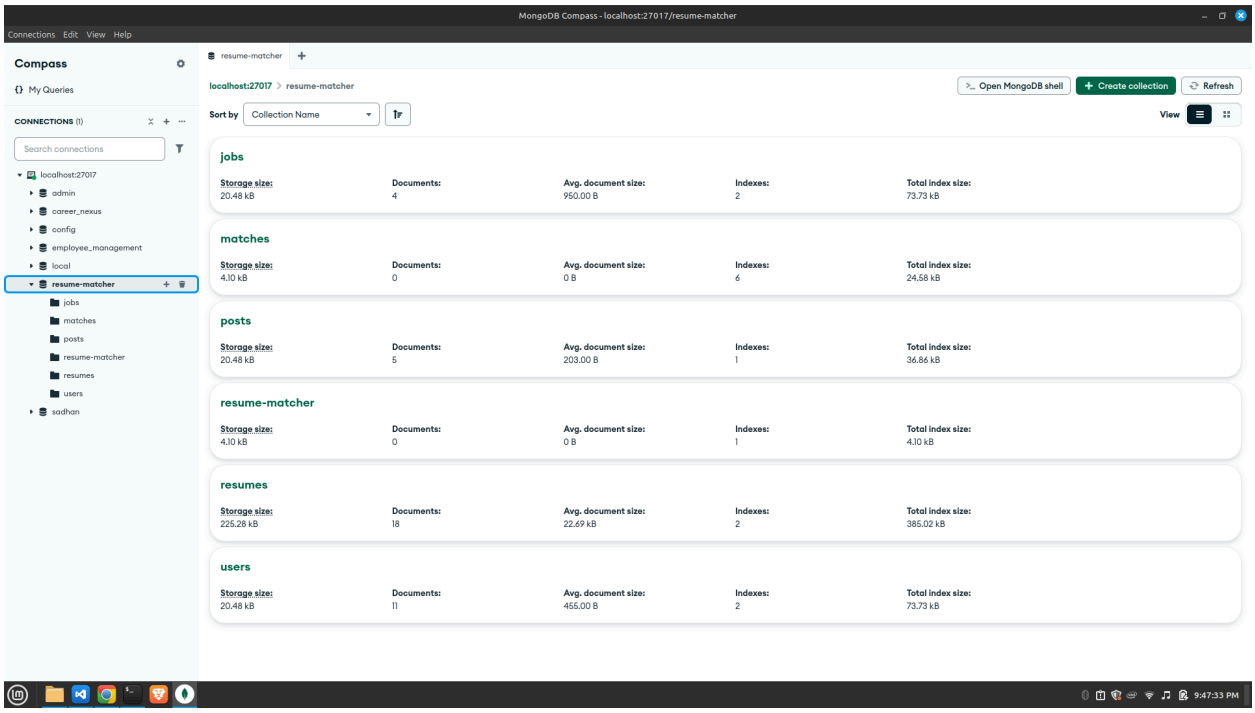


Figure 32:Database Collection

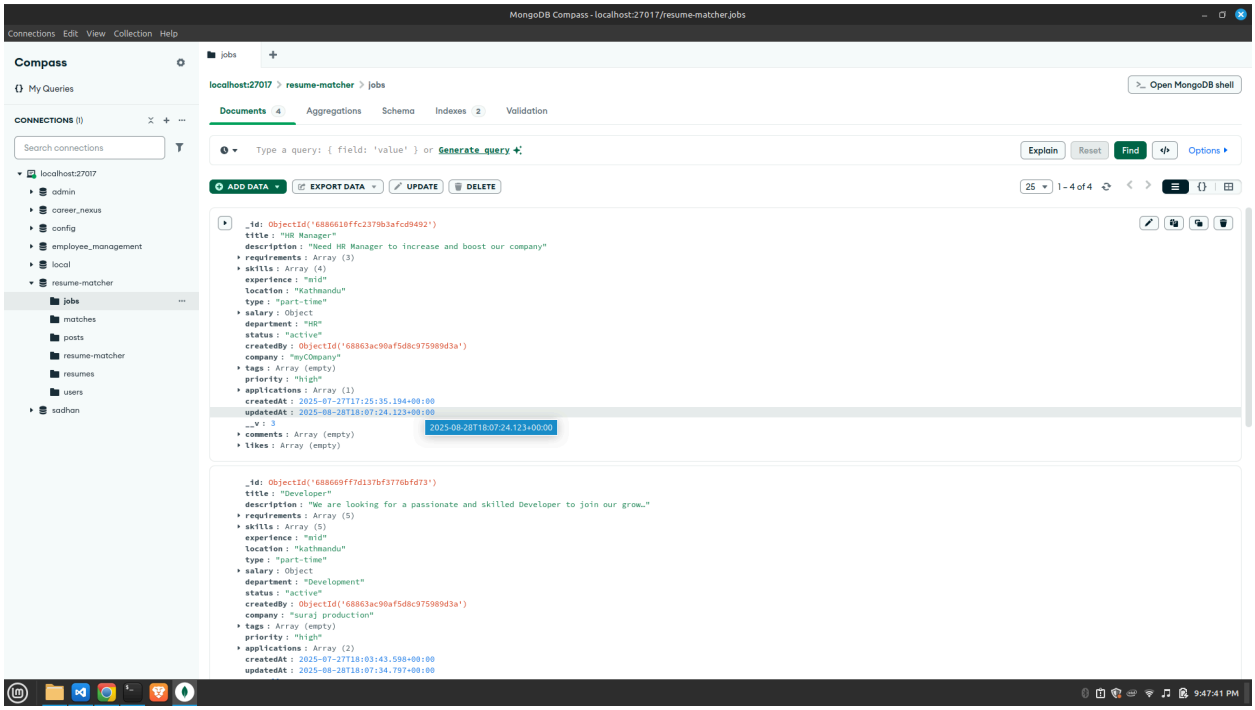


Figure 33:Jobs Collection

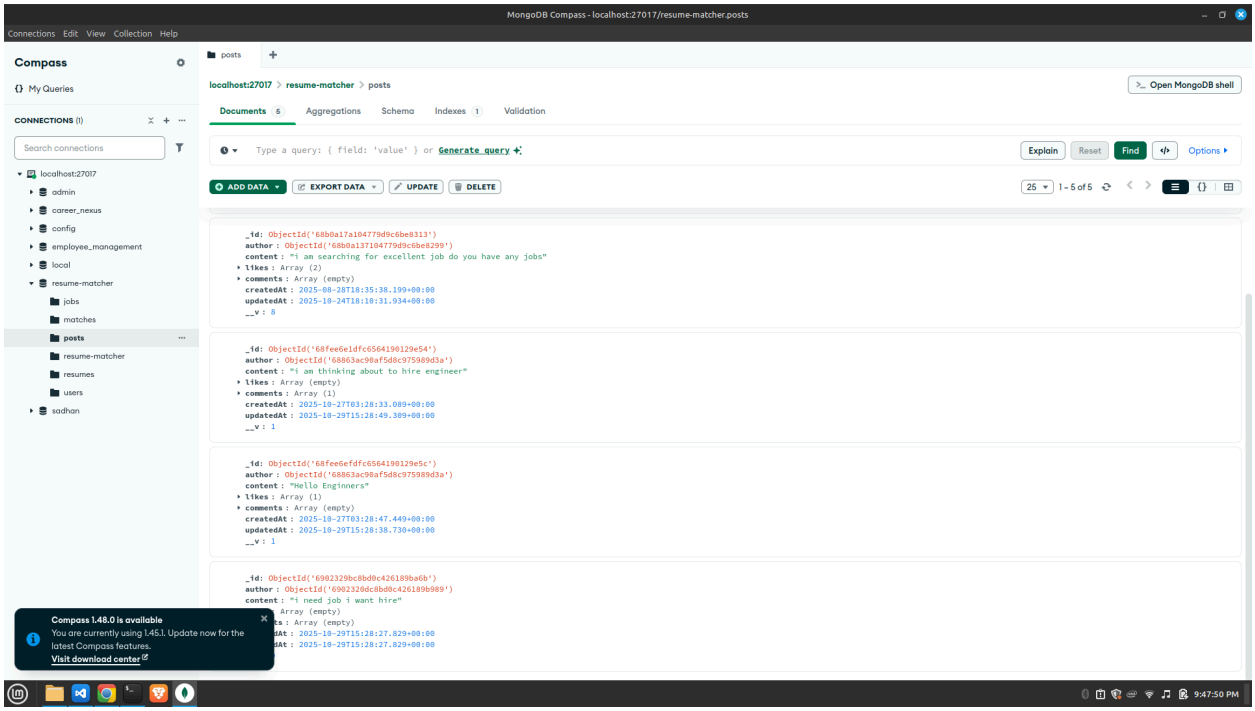


Figure 34:Posts Collection

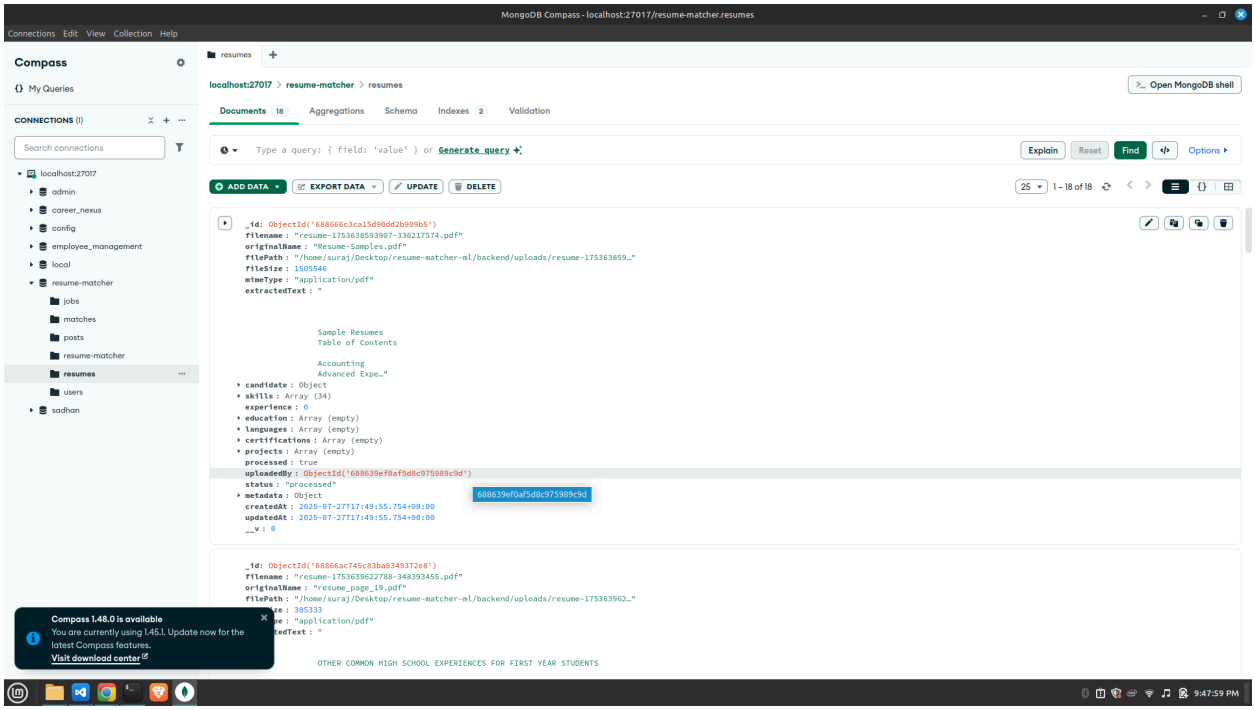


Figure 35:Resume Collection

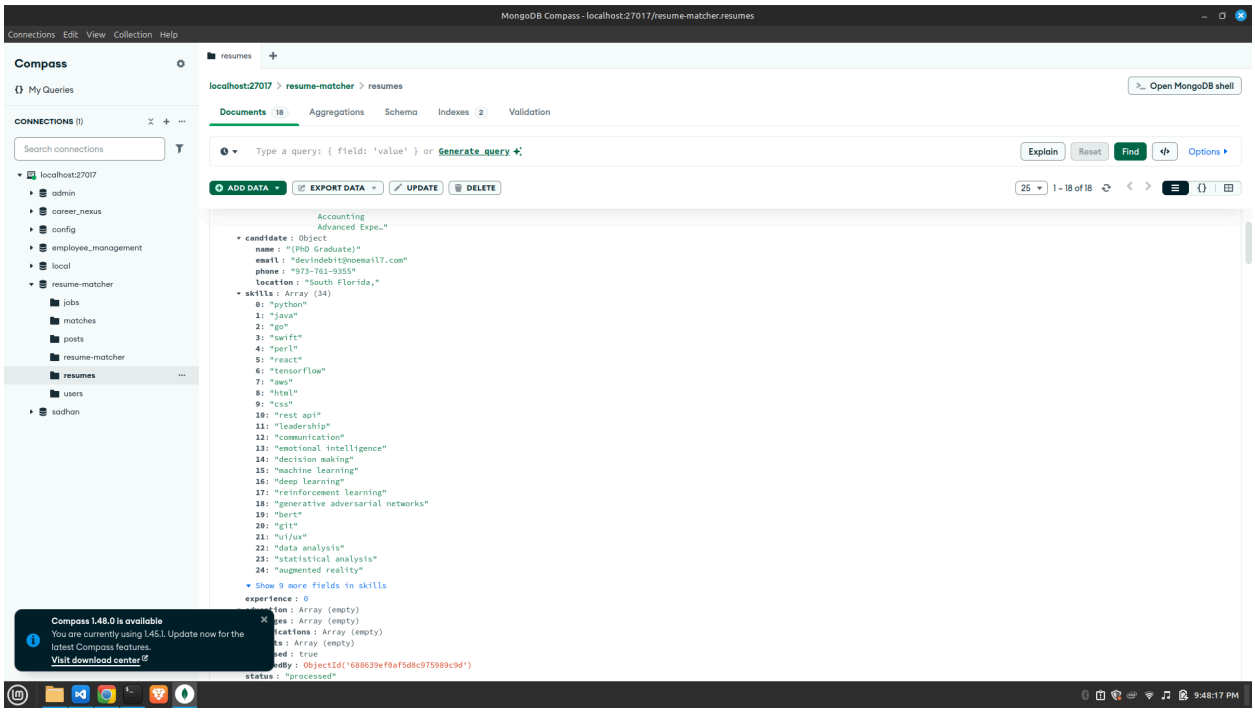


Figure 36:User Collection

Log of visits to Supervisor

Table 10: Supervisor Meet Log

Date	**Remarks**
2082-04-11	Initial Proposal submitted for review and supervision to College
2082-04-14	Initial Proposal submitted to Supervisor for review and supervision
2082-04-19	Project Discussion and showed Figma UI/UX for the project
2082-05-11	Submitted final defense report for review and supervision.
2082-05-17	Resubmitted final defense report for review and supervision.
2082-07-12	Resubmitted final defense report and Presentation for review and supervision.